

第6章 RAG 和智能体

为了解决任务，模型既需要关于如何完成任务的指令，也需要必要的信息。就像人类在缺乏信息时更可能给出错误答案一样，AI模型在缺少上下文时更容易犯错和产生幻觉。对于给定应用，模型的指令对所有查询都是通用的，而上下文则针对每个查询而特定。上一章讨论了如何为模型编写良好的指令。本章将重点讨论如何为每个查询构建相关上下文。

上下文构建的两种主导模式是RAG(检索增强生成)和智能体。RAG模式允许模型从外部数据源检索相关信息。智能体模式则允许模型使用网络搜索和新闻API等工具收集信息。

虽然RAG模式主要用于构建上下文，但智能体模式的功能远不止于此。外部工具可以帮助模型弥补其不足并扩展其能力。最重要的是，工具赋予模型直接与世界互动的能力，使其能够自动化我们生活的许多方面。

RAG和智能体模式之所以令人兴奋，是因为它们为已经强大的模型带来了新的能力。在短时间内，它们已经成功捕获了集体想象力，产生了令人难以置信的演示和产品，使许多人相信它们代表着未来。本章将详细介绍这些模式的工作原理及其前景。

RAG

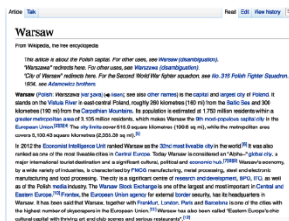
RAG是一种通过从外部存储源检索相关信息来增强模型生成能力的技术。外部存储源可以是内部数据库、用户的以前的聊天会话或互联网。

检索再生成模式最早在“通过阅读维基百科回答开放域问题”(Chen等, 2017)中引入。在这项工作中，系统首先检索与问题最相关的五个维基百科页面，然后模型使用或阅读这些页面中的信息生成答案，如图6-1所示。

Q: How many of Warsaw's inhabitants spoke Polish in 1933?

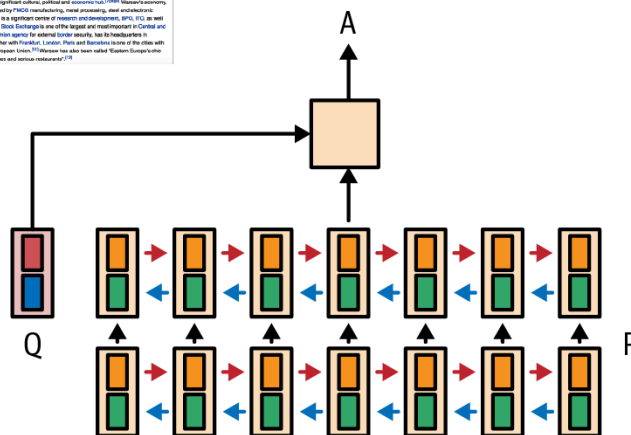


Document
retriever



Document
reader

833,500



"检索增强生成"一词是在"面向知识密集型NLP任务的检索增强生成"(Lewis等, 2020)中提出的。该论文提出RAG作为知识密集型任务的解决方案, 其中所有可用知识无法直接输入到模型中。通过RAG, 只有由检索器确定的与查询最相关的信息才会被检索并输入到模型中。Lewis等发现, 访问相关信息可以帮助模型生成更详细的响应, 同时减少幻觉。

例如, 对于查询"Acme的fancy-printer-A300能打印100pps吗?", 如果模型获得了fancy-printer-A300的规格, 它将能够更好地回应。

你可以将RAG视为一种为每个查询构建特定上下文的技术, 而不是对所有查询使用相同的上下文。这有助于管理用户数据, 因为它允许你仅在与特定用户相关的查询中包含该用户的特定数据。

基础模型的上下文构建相当于经典机器学习模型的特征工程。它们服务于相同的目的: 给模型提供处理输入所需的必要信息。

在基础模型的早期, RAG作为最常见的模式之一出现。其主要目的是克服模型的上下文限制。许多人认为足够长的上下文将终结RAG。我不这么认为。首先, 无论模型的上下文长度有多长, 总会有需要比这更长上下文的应用。毕竟, 可用数据量只会随时间增长。人们会生成和添加新数据, 但很少删除数据。上下文长度正在迅速扩展, 但对于任意应用的数据需求而言, 扩展速度还不够快。

其次, 能够处理长上下文的模型并不一定能够很好地利用这些上下文, 正如"上下文长度和上下文效率"中所讨论的。上下文越长, 模型越可能关注上下文的错误部分。每增加一个上下文令牌都会

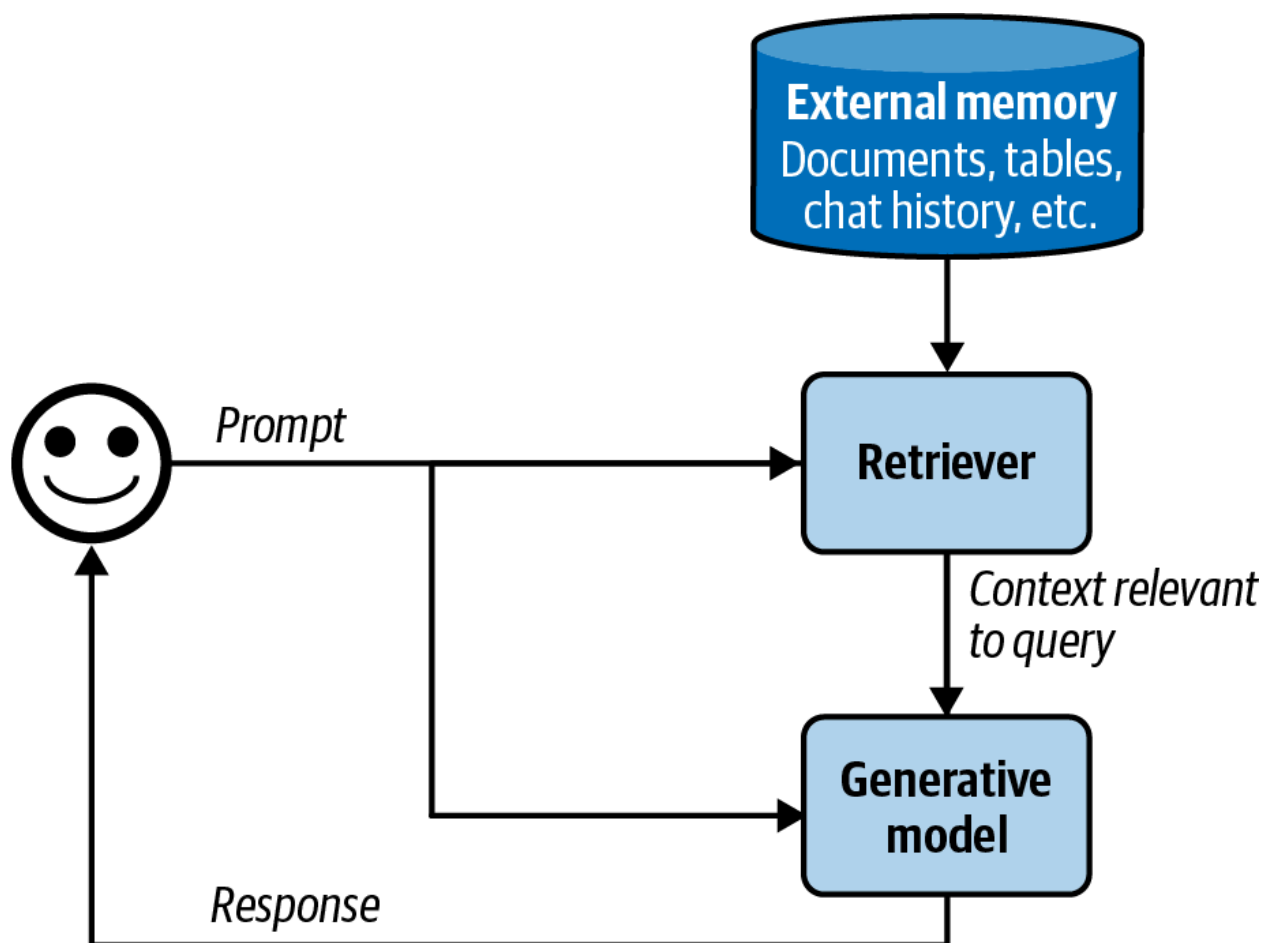
增加额外成本，并可能增加额外延迟。RAG允许模型仅使用每个查询最相关的信息，减少输入令牌数量，同时可能提高模型性能。

扩展上下文长度的努力正与使模型更有效地使用上下文的努力同时进行。我不会感到惊讶，如果某个模型提供商整合检索类或注意力类机制，帮助模型挑选出上下文中最突出的部分来使用。

注意 Anthropic建议，对于Claude模型，如果"你的知识库小于200,000个令牌(约500页材料)，你可以直接将整个知识库包含在给模型的提示中，无需RAG或类似方法"(Anthropic, 2024)。如果其他模型开发者也能为他们的模型提供类似的RAG与长上下文使用指南，将会非常棒。

RAG架构

RAG系统有两个组件：从外部存储源检索信息的检索器和基于检索信息生成响应的生成器。图6-2显示了RAG系统的高级架构。



在原始RAG论文中, Lewis等人一起训练了检索器和生成模型。在当今的RAG系统中, 这两个组件通常是分开训练的, 许多团队使用现成的检索器和模型构建他们的RAG系统。然而, 对整个RAG系统进行端到端的微调可以显著提高其性能。

RAG系统的成功取决于其检索器的质量。检索器有两个主要功能: 索引和查询。索引涉及处理数据, 使其稍后可以快速检索。发送查询以检索与之相关的数据称为查询。如何索引数据取决于你稍后想如何检索它。

现在我们已经介绍了主要组件, 让我们考虑RAG系统如何工作的例子。为简单起见, 假设外部存储是文档数据库, 如公司的备忘录、合同和会议记录。文档可以是10个令牌或100万个令牌。简单地检索整个文档可能导致上下文任意长。为避免这种情况, 可以将每个文档分成更易管理的块。分块策略将在本章后面讨论。现在, 假设所有文档已被分成可行的块。对于每个查询, 我们的目标是检索与该查询最相关的数据块。通常需要少量后处理来将检索到的数据块与用户提示连接起来生成最终提示。这个最终提示然后被输入到生成模型中。

注意 在本章中, 我使用术语"文档"同时指代"文档"和"块", 因为从技术上讲, 文档的一个块也是一个文档。这样做是为了保持本书的术语与经典NLP和信息检索(IR)术语一致。

检索算法

检索并非RAG独有。信息检索是一个有着百年历史的概念。它是搜索引擎、推荐系统、日志分析等的基础。为传统检索系统开发的许多检索算法也可用于RAG。例如, 信息检索是一个肥沃的研究领域, 有大量支持产业, 很难在几页内充分涵盖。因此, 本节将只介绍大致情况。有关信息检索的更深入资源, 请参阅本书的GitHub存储库。

注意 检索通常限于一个数据库或系统, 而搜索涉及跨各种系统的检索。本章中交替使用检索和搜索这两个术语。

本质上, 检索通过根据文档与给定查询的相关性对文档进行排名来工作。检索算法根据相关性分数的计算方式而不同。我将从两种常见的检索机制开始: 基于术语的检索和基于嵌入的检索。

稀疏与密集检索

在文献中, 你可能会遇到将检索算法分为以下类别的划分: 稀疏与密集。然而, 本书选择了基于术语与基于嵌入的分类。

稀疏检索器使用稀疏向量表示数据。稀疏向量是一个大多数值为0的向量。基于术语的检索被认为是稀疏的, 因为每个术语可以使用稀疏的独热向量表示, 这是一个除一个值为1外其他位置都为0的向量。向量大小是词汇表的长度。值为1的位置对应于该术语在词汇表中的索引。

如果我们有一个简单的字典, `{"food": 0, "banana": 1, "slug": 2}`, 那么"food"、"banana"和"slug"的独热向量分别是`[1, 0, 0]`、`[0, 1, 0]`和`[0, 0, 1]`。

密集检索器使用密集向量表示数据。密集向量是一个大多数值不为0的向量。基于嵌入的检索通常被认为是密集的，因为嵌入通常是密集向量。然而，也有稀疏嵌入。例如，SPLADE(稀疏词汇和扩展)是一种使用稀疏嵌入工作的检索算法(Foral等, 2021)。它利用由BERT生成的嵌入，但使用正则化将大多数嵌入值推向0。这种稀疏性使嵌入操作更高效。

稀疏与密集的划分导致SPLADE与基于术语的算法被归为一组，尽管SPLADE的操作、优势和劣势与基于嵌入的检索相比，更类似于密集嵌入检索，而不是基于术语的检索。基于术语与基于嵌入的划分避免了这种错误分类。

基于术语的检索

给定查询，查找相关文档最直接的方法是使用关键词。有些人称这种方法为词汇检索。例如，给定查询"AI工程"，模型将检索所有包含"AI工程"的文档。然而，这种方法有两个问题：

1. 许多文档可能包含给定术语，而模型可能没有足够的上下文空间将所有这些文档作为上下文包含进来。一种启发式方法是包含包含该术语最多次数的文档。假设是，术语在文档中出现的次数越多，该文档与该术语越相关。术语在文档中出现的次数称为术语频率(TF)。
2. 提示可能很长且包含许多术语。有些比其他更重要。例如，提示"在家烹饪越南食品的易于遵循的食谱"包含九个术语：easy-to-follow、recipes、for、vietnamese、food、to、cook、at、home。你希望关注更具信息性的术语，如vietnamese和recipes，而不是for和at。你需要一种方法来识别重要术语。
一种直觉是，包含某个术语的文档越多，该术语的信息量越少。"For"和"at"可能出现在大多数文档中，因此，它们的信息量较少。所以术语的重要性与它出现在的文档数量成反比。这个指标称为逆文档频率(IDF)。要计算术语的IDF，计算包含该术语的所有文档数量，然后用文档总数除以这个数量。如果有10个文档，其中5个包含给定术语，那么该术语的IDF为 $10 / 5 = 2$ 。术语的IDF越高，它就越重要。

TF-IDF是一种结合这两个指标的算法：术语频率(TF)和逆文档频率(IDF)。在数学上，文档D对查询Q的TF-IDF分数计算如下：

- 假设 $t_1, t_2, \dots, t_n$ 是查询Q中的术语。
- 给定术语 t ，该术语在文档D中的术语频率为 $f(t, D)$ 。
- 设N为文档总数， $C(t)$ 为包含 t 的文档数。术语 t 的IDF值可以写为 $\log(N/C(t))$ 。
- 简单地说，文档D相对于Q的TF-IDF分数定义为 $\sum f(t_i, D) \times \log(N/C(t_i))$ 。

两种常见的基于术语的检索解决方案是Elasticsearch和BM25。Elasticsearch(Shay Banon, 2010)建立在Lucene之上，使用称为倒排索引的数据结构。这是一个从术语映射到包含它们的文档的字典。该字典允许在给定术语时快速检索文档。索引还可能存储额外信息，如术语频率和文档计数(包含该术语的文档数量)，这对计算TF-IDF分数很有帮助。表6-1说明了倒排索引。

表6-1。倒排索引的简化示例

术语	文档计数	(文档索引, 术语频率) 对于包含该术语的所有文档
banana	2	(10, 3), (5, 2)
machine	4	(1, 5), (10, 1), (38, 9), (42, 5)
learning	3	(1, 5), (38, 7), (42, 5)
...	...	...

Okapi BM25, 最佳匹配算法的第25代, 由Robertson等人在1980年代开发。其评分器是TF-IDF的修改版。与简单的TF-IDF相比, BM25按文档长度归一化术语频率分数。较长的文档更可能包含给定术语并具有更高的术语频率值。

BM25及其变体(BM25+, BM25F)在行业中仍被广泛使用, 并作为与现代、更复杂的检索算法(如下面讨论的基于嵌入的检索)比较的强大基准。

我略过了一个过程, 即分词, 将查询分解为单个术语的过程。最简单的方法是将查询分成单词, 将每个单词视为单独的术语。然而, 这可能导致多词术语被分解为单个单词, 失去其原始含义。例如, "hot dog"将被分为"hot"和"dog"。发生这种情况时, 两者都不保留原始术语的含义。缓解这个问题的一种方法是将最常见的n-gram视为术语。如果二元组"hot dog"很常见, 它将被视为一个术语。

此外, 你可能希望将所有字符转换为小写, 删除标点符号, 并消除停用词(如"the"、"and"、"is"等)。基于术语的检索解决方案通常会自动处理这些问题。经典的NLP包, 如NLTK(自然语言工具包)、spaCy和斯坦福的CoreNLP, 也提供分词功能。

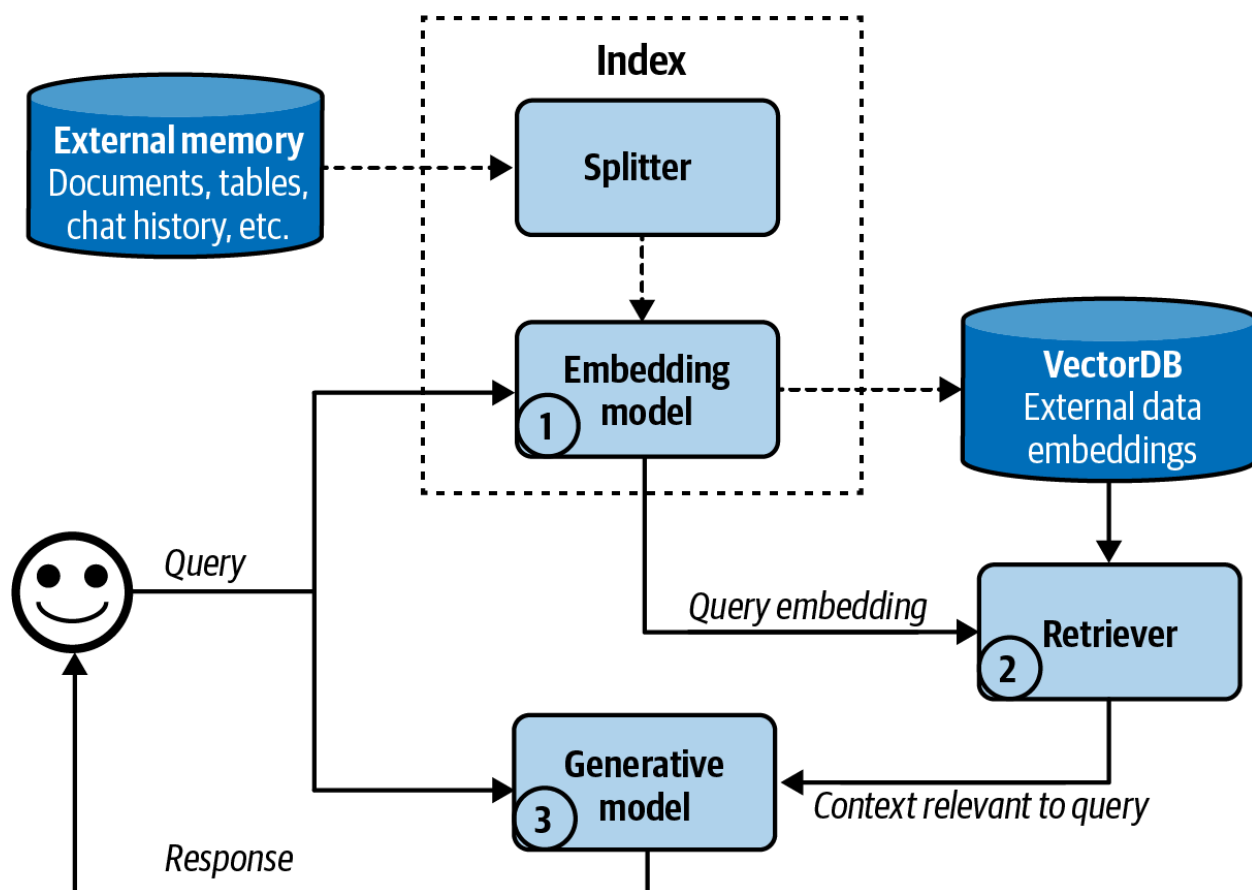
第4章讨论了基于两个文本的n-gram重叠来测量它们之间的词汇相似性。我们可以根据文档与查询的n-gram重叠程度来检索文档吗? 是的, 我们可以。当查询和文档长度相似时, 这种方法效果最好。如果文档比查询长得多, 它们包含查询n-gram的可能性会增加, 导致许多文档具有相似的高重叠分数。这使得难以区分真正相关的文档和不太相关的文档。

基于嵌入的检索

基于术语的检索在词汇层面而非语义层面计算相关性。正如第3章中提到的, 文本的外观并不一定能捕捉其含义。这可能导致返回与你的意图无关的文档。例如, 查询"transformer架构"可能返回有关电气设备或《变形金刚》电影的文档。另一方面, 基于嵌入的检索器旨在根据文档的含义与查询的匹配程度对文档进行排名。这种方法也被称为语义检索。

使用基于嵌入的检索, 索引有一个额外功能: 将原始数据块转换为嵌入。存储生成的嵌入的数据库称为向量数据库。查询然后由两个步骤组成, 如图6-3所示:

1. 嵌入模型:使用与索引期间相同的嵌入模型将查询转换为嵌入。
2. 检索器:获取k个数据块,其嵌入与查询嵌入最接近,由检索器确定。要获取的数据块数量k取决于用例、生成模型和查询。



这里显示的基于嵌入的检索工作流程是简化的。实际语义检索系统可能包含其他组件,如重排器以重新排列所有检索到的候选项,以及缓存以减少延迟。

使用基于嵌入的检索,我们再次遇到第3章中讨论的嵌入。提醒一下,嵌入通常是一个向量,旨在保留原始数据的重要属性。如果嵌入模型不好,基于嵌入的检索器将无法工作。

基于嵌入的检索还引入了一个新组件:向量数据库。向量数据库存储向量。然而,存储是向量数据库的简单部分。困难的部分是向量搜索。给定查询嵌入,向量数据库负责在数据库中查找接近查询的向量并返回它们。必须以一种使向量搜索快速高效的方式索引和存储向量。

与生成式AI应用依赖的许多其他机制一样,向量搜索并非生成式AI独有。向量搜索在任何使用嵌入的应用中都很常见:搜索、推荐、数据组织、信息检索、聚类、欺诈检测等。

向量搜索通常被框架为最近邻搜索问题。例如,给定查询,找到k个最近的向量。简单解决方案是k最近邻(k-NN),其工作方式如下:

1. 使用诸如余弦相似度之类的度量计算查询嵌入与数据库中所有向量之间的相似度分数。
2. 按相似度分数对所有向量进行排名。
3. 返回相似度分数最高的k个向量。

这种简单解决方案确保结果精确，但计算量大且速度慢。它应该只用于小型数据集。

对于大型数据集，向量搜索通常使用近似最近邻(ANN)算法完成。由于向量搜索的重要性，已经为其开发了许多算法和库。一些流行的向量搜索库有FAISS(Facebook AI相似度搜索)(Johnson等, 2017)、Google的ScaNN(可扩展最近邻)(Sun等, 2020)、Spotify的Annoy(Bernhardsson, 2013)和Hnswlib(分层导航小世界)(Malkov和Yashunin, 2016)。

大多数应用开发者不会自己实现向量搜索，所以我只会对不同方法进行简短概述。这个概述可能在评估解决方案时有所帮助。

一般来说，向量数据库将向量组织成桶、树或图。向量搜索算法根据它们使用的启发式方法不同，以增加类似向量彼此接近的可能性。向量也可以被量化(降低精度)或变得稀疏。这样做是因为量化和稀疏向量的计算强度较低。对于那些想了解更多关于向量搜索的信息的人，Zilliz有一个关于它的出色系列。以下是一些重要的向量搜索算法：

LSH(局部敏感哈希)(Indyk和Motwani, 1999) 这是一种强大且多功能的算法，不仅适用于向量。这涉及将类似向量哈希到相同的桶中以加速相似性搜索，以效率换取一些准确性。它在FAISS和Annoy中实现。

HNSW(分层导航小世界)(Malkov和Yashunin, 2016) HNSW构建一个多层图，其中节点表示向量，边连接相似向量，通过遍历图边允许最近邻搜索。其作者的实现是开源的，它也在FAISS和Milvus中实现。

产品量化(Jégou等, 2011) 这通过将每个向量分解为多个子向量，将每个向量降维为更简单的低维表示。然后使用低维表示计算距离，这样计算速度快得多。产品量化是FAISS的关键组成部分，几乎所有流行的向量搜索库都支持它。

IVF(倒排文件索引)(Sivic和Zisserman, 2003) IVF使用K-means聚类将相似向量组织到同一个集群中。根据数据库中向量的数量，通常将聚类数量设置为每个聚类平均有100到10,000个向量。在查询期间，IVF找到与查询嵌入最接近的聚类中心，这些聚类中的向量成为候选邻居。与产品量化一起，IVF形成了FAISS的骨干。

Annoy(近似最近邻, Oh Yeah)(Bernhardsson, 2013) Annoy是一种基于树的方法。它构建多个二叉树，每棵树使用随机标准将向量分成聚类，例如随机绘制一条线并使用这条线将向量分成两个分支。在搜索期间，它遍历这些树以收集候选邻居。Spotify已开源其实现。

还有其他算法，如微软的SPTAG(空间分区树和图)和FLANN(快速近似最近邻库)。

虽然向量数据库随着RAG的兴起成为了自己的类别，但任何可以存储向量的数据库都可以被称为向量数据库。许多传统数据库已经或将要扩展以支持向量存储和向量搜索。

比较检索算法

由于检索历史悠久，其许多成熟解决方案使基于术语和基于嵌入的检索都相对容易开始。每种方法都有其优缺点。

基于术语的检索在索引和查询期间通常比基于嵌入的检索快得多。术语提取比嵌入生成更快，从术语映射到包含它的文档的计算量可能比最近邻搜索少。

基于术语的检索也能很好地开箱即用。像Elasticsearch和BM25这样的解决方案已经成功地支持了许多搜索和检索应用。然而，其简单性也意味着它具有较少的可调整组件来提高其性能。

另一方面，基于嵌入的检索可以随着时间的推移显著改进，以胜过基于术语的检索。你可以微调嵌入模型和检索器，可以单独微调，一起微调，或与生成模型一起微调。然而，将数据转换为嵌入可能会模糊关键词，例如特定错误代码(如EADDRNOTAVAIL(99))或产品名称，使它们以后更难搜索。通过结合基于嵌入的检索和基于术语的检索，可以解决这一限制，将在本章后面讨论。

检索器的质量可以根据它检索的数据质量进行评估。RAG评估框架常用的两个指标是上下文精度和上下文召回率，或简称精度和召回率(上下文精度也称为上下文相关性)：

上下文精度 在所有检索的文档中，有多少百分比与查询相关？

上下文召回率 在所有与查询相关的文档中，检索了多少百分比？

要计算这些指标，你需要策划一个评估集，其中包含测试查询列表和一组文档。对于每个测试查询，你对每个测试文档进行注释，标记其是否相关。注释可以由人类或AI评判完成。然后你在此评估集上计算检索器的精度和召回率分数。

在生产中，一些RAG框架只支持上下文精度，而不支持上下文召回率。要计算给定查询的上下文召回率，你需要注释数据库中所有文档与该查询的相关性。上下文精度更简单计算。你只需要将检索到的文档与查询进行比较，这可以由AI评判完成。

如果你关心检索文档的排名，例如，更相关的文档应该排在前面，你可以使用如NDCG(归一化折扣累积增益)、MAP(平均精度均值)和MRR(平均倒数排名)等指标。

对于语义检索，你还需要评估嵌入的质量。如第3章所述，嵌入可以独立评估——如果更相似的文档具有更接近的嵌入，则被认为是好的。嵌入也可以通过它们在特定任务中的表现来评估。MTEB基准(Muennighoff等，2023)评估嵌入在广泛任务范围内的表现，包括检索、分类和聚类。

检索器的质量也应该在整个RAG系统的上下文中进行评估。最终，如果检索器有助于系统生成高质量答案，那么它就是好的。评估生成模型输出的内容在第3章和第4章中讨论。

语义检索系统的性能承诺是否值得追求取决于你对成本和延迟的重视程度，特别是在查询阶段。由于RAG延迟的很大部分来自输出生成，特别是对于长输出，查询嵌入生成和向量搜索增加的延迟可能与RAG总延迟相比微不足道。即使如此，增加的延迟仍然可能影响用户体验。

另一个关注点是成本。生成嵌入需要花钱。如果你的数据频繁变化并需要频繁重新生成嵌入，这尤其是个问题。想象一下每天必须为1亿个文档生成嵌入！根据你使用的向量数据库，向量存储和向量搜索查询也可能很昂贵。看到公司在向量数据库上的支出是其在模型API上支出的五分之一甚至一半并不罕见。

表6-2显示了基于术语的检索和基于嵌入的检索在速度、性能和成本方面的并排比较。

表6-2. 基于术语的检索和语义检索在速度、性能和成本方面的比较

	基于术语的检索	基于嵌入的检索
查询速度	比基于嵌入的检索快得多	查询嵌入生成和向量搜索可能很慢
性能	通常开箱即有强大性能，但难以改进；由于术语歧义可能检索错误文档	通过微调可以胜过基于术语的检索；专注于语义而非术语，允许使用更自然的查询
成本	比基于嵌入的检索便宜得多	嵌入、向量存储和向量搜索解决方案可能很昂贵

在检索系统中，你可以在索引和查询之间做出特定权衡。索引越详细，检索过程就越精确，但索引过程会更慢且更消耗内存。想象一下建立潜在客户的索引。添加更多细节(如姓名、公司、电子邮件、电话、兴趣)使找到相关人员更容易，但构建时间更长且需要更多存储空间。

通常，详细索引如HNSW提供高准确性和快速查询时间，但需要大量时间和内存来构建。相比之下，像LSH这样的简单索引创建更快、内存消耗更少，但查询更慢且准确性更低。

ANN-Benchmarks网站使用四个主要指标比较不同ANN算法在多个数据集上的表现，考虑了索引和查询之间的权衡。这些包括：

召回率
算法找到的最近邻居的比例。

每秒查询数(QPS)
算法每秒可以处理的查询数。这对高流量应用至关重要。

构建时间

构建索引所需的时间。如果需要频繁更新索引(例如, 因为数据变化), 这个指标尤为重要。

索引大小

算法创建的索引大小, 对评估其可扩展性和存储需求至关重要。

此外, BEIR(信息检索基准测试) (Thakur等, 2021)是检索评估工具。它支持跨14个常见检索基准的检索系统。

总结一下, RAG系统的质量应该逐组件和端到端进行评估。为此, 你应该做以下几件事:

1. 评估检索质量。
2. 评估最终RAG输出。
3. 评估嵌入(对于基于嵌入的检索)。

组合检索算法

鉴于不同检索算法的明显优势, 生产环境中的检索系统通常会结合多种方法。结合基于术语的检索和基于嵌入的检索称为混合搜索。

不同算法可以按顺序使用。首先, 一个便宜但精度较低的检索器(如基于术语的系统) 获取候选项。然后, 一个更精确但更昂贵的机制(如k-最近邻) 找出这些候选项中最好的。第二步也称为重排序。

例如, 给定术语"transformer", 你可以获取所有包含单词transformer的文档, 无论它们是关于电气设备、神经网络架构还是电影。然后使用向量搜索在这些文档中找到与你的transformer查询实际相关的文档。另一个例子, 考虑查询"谁负责对X的最多销售?"首先, 你可能使用关键词X获取所有与X相关的文档。然后, 使用向量搜索检索与"谁负责最多销售?"相关的上下文。

不同算法也可以并行用作集成。记住, 检索器通过按查询相关性对文档进行排名工作。你可以同时使用多个检索器获取候选项, 然后将这些不同排名组合在一起生成最终排名。

组合不同排名的算法称为互惠排名融合(RRF)(Cormack等, 2009)。它根据文档被检索器排名的位置为每个文档分配分数。直观上, 如果排名第一, 其分数为 $1/1 = 1$ 。如果排名第二, 其分数为 $1/2 = 0.5$ 。排名越高, 分数越高。

文档的最终分数是其对所有检索器的分数之和。如果一个文档被一个检索器排在第一位, 被另一个检索器排在第二位, 其分数为 $1 + 0.5 = 1.5$ 。这个例子是对RRF的过度简化, 但它展示了基础知识。文档D的实际公式更复杂, 如下所示:

$$\text{Score}(D) = \sum_{i=1}^n \frac{1}{k + r_i(D)}$$

n 是排名列表的数量;每个排名列表由一个检索器生成。 $\text{rank}_i(D)$ 是文档由检索器 i 排名的位置。 k 是一个常数,用于避免除以零并控制较低排名文档的影响。 k 的典型值为60。

检索优化

根据任务不同,某些策略可以增加相关文档被获取的机会。这里讨论的四种策略是分块策略、重排序、查询重写和上下文检索。

分块策略

如何索引数据取决于你以后打算如何检索它。上一节介绍了不同的检索算法及其各自的索引策略。那里的讨论基于文档已经被分成可管理块的假设。在本节中,我将介绍不同的分块策略。这是一个重要考虑因素,因为你使用的分块策略可能对检索系统性能产生重大影响。

最简单的策略是根据特定单位将文档分成相等长度的块。常见单位是字符、单词、句子和段落。例如,你可以将每个文档分成2,048个字符或512个单词的块。你也可以分割每个文档,使每个块包含固定数量的句子(如20个句子)或段落(如每个段落是其自己的块)。

你也可以使用越来越小的单位递归分割文档,直到每个块符合你的最大块大小。例如,你可以先将文档分成小节。如果一个小节太长,将其分成段落。如果一个段落仍然太长,将其分成句子。这减少了相关文本被任意截断的可能性。

特定文档也可能支持创新的分块策略。例如,有专门为不同编程语言开发的分割器。问答文档可以按问题或答案对分割,每对构成一个块。中文文本可能需要与英文文本不同的分割方式。

当文档被分成无重叠的块时,这些块可能在重要上下文中间被切断,导致关键信息丢失。考虑文本"I left my wife a note"。如果它被分成"I left my wife"和"a note",这两个块都不能传达原始文本的关键信息。重叠确保重要的边界信息至少包含在一个块中。如果你将块大小设为2,048个字符,你可以将重叠大小设为20个字符。

块大小不应超过生成模型的最大上下文长度。对于基于嵌入的方法,块大小也不应超过嵌入模型的上下文限制。

你也可以使用由生成模型的分词器确定的标记作为单位分块文档。假设你想使用Llama 3作为你的生成模型。然后首先使用Llama 3的分词器对文档进行分词。然后可以使用标记作为边界将文档分成块。按标记分块使与下游模型一起工作变得更容易。然而,这种方法的缺点是,如果你切换到使用不同分词器的另一个生成模型,你需要重新索引数据。

无论选择哪种策略，块大小都很重要。较小的块大小允许更多样化的信息。较小的块意味着你可以在模型的上下文中放入更多块。如果将块大小减半，可以容纳两倍多的块。更多块可以为模型提供更广泛的信息范围，使模型能够产生更好的答案。

然而，小块大小可能导致重要信息丢失。想象一个文档在整个文档中包含关于主题X的重要信息，但X只在文档前半部分提到。如果将此文档分成两块，文档的后半部分可能不会被检索，模型将无法使用其信息。

较小的块大小也会增加计算开销。这对基于嵌入的检索特别是个问题。将块大小减半意味着需要索引两倍多的块，生成和存储两倍多的嵌入向量。向量搜索空间将变为两倍大，这可能会降低查询速度。

没有普遍最佳的块大小或重叠大小。你必须通过实验找到最适合你的方法。

重排序

检索器生成的初始文档排名可以进一步重排序以更加准确。重排序对减少检索文档数量特别有用，无论是为了将它们放入模型的上下文中，还是为了减少输入标记的数量。

重排序的一种常见模式在"组合检索算法"中讨论。一个便宜但精度较低的检索器获取候选项，然后一个更精确但更昂贵的机制重排这些候选项。

文档也可以基于时间重排序，对更近期的数据给予更高权重。这对时间敏感的应用如新闻聚合、与你的电子邮件聊天(例如，可以回答关于你的电子邮件问题的聊天机器人)或股市分析很有用。

上下文重排序与传统搜索重排序的不同之处在于项目的确切位置不那么关键。在搜索中，排名(例如，第一或第五)至关重要。在上下文重排序中，文档的顺序仍然重要，因为它影响模型如何处理它们。正如"上下文长度和上下文效率"中所讨论的，模型可能更好地理解上下文开头和结尾的文档。然而，只要包含文档，其顺序的影响与搜索排名相比不那么显著。

查询重写

查询重写也称为查询重新表述、查询规范化，有时也称为查询扩展。考虑以下对话：

User: When was the last time John Doe bought something from us?
AI: John last bought a Fruity Fedora hat from us two weeks ago, on January 3, 2030.
User: How about Emily Doe?

最后一个问题"Emily Doe呢?"在没有上下文的情况下是不明确的。如果你逐字使用此查询检索文档，可能会得到不相关的结果。你需要重写此查询以反映用户实际在询问什么。新查询应该独立有意义。在这种情况下，查询应重写为"Emily Doe上次从我们这里买东西是什么时候?"

虽然我将查询重写放在"RAG"中,但查询重写并非RAG独有。在传统搜索引擎中,查询重写通常使用启发式方法完成。在AI应用中,查询重写也可以使用其他AI模型完成,使用类似"给定以下对话,重写最后一个用户输入以反映用户实际在询问什么"的提示。图6-4显示了ChatGPT如何使用这个提示重写查询。

Given the following conversation, rewrite the last user input to reflect what the user is actually asking.

User: When was the last time John Doe bought something from us?

AI: John last bought a Fruity Fedora hat from us two weeks ago, on January 3, 2030.

User: How about Emily Doe?

When was the last time Emily Doe bought something from us?

如果你需要进行身份解析或合并其他知识,查询重写可能会变得复杂。例如,如果用户问"他的妻子呢?",你首先需要查询数据库找出他的妻子是谁。如果你没有这些信息,重写模型应该承认这个查询无法解决,而不是虚构一个名字,导致错误答案。

上下文检索

上下文检索的理念是用相关上下文增强每个块,使检索相关块更容易。一种简单技术是用元数据如标签和关键词增强块。对于电子商务,产品可以用其描述和评论增强。图像和视频可以通过它们的标题或说明进行查询。

元数据还可能包括从块中自动提取的实体。如果文档包含特定术语如错误代码EADDRNOTAVAIL(99),将它们添加到文档的元数据中可以让系统通过该关键词检索它,即使在文档已转换为嵌入之后。

你还可以用它来回答的问题增强每个块。对于客户支持,你可以用相关问题增强每篇文章。例如,关于如何重置密码的文章可以增强为包含查询如"如何重置密码?"、"我忘记了密码"、"我无法登录",甚至"帮助,我找不到我的账户"。

如果一个文档被分成多个块,某些块可能缺乏必要的上下文来帮助检索器理解块的内容。为避免这种情况,你可以用原始文档的上下文增强每个块,如原始文档的标题和摘要。Anthropic使用AI模型生成一个简短的上下文,通常50-100个令牌,解释块及其与原始文档的关系。以下是Anthropic用于此目的的提示(Anthropic, 2024):

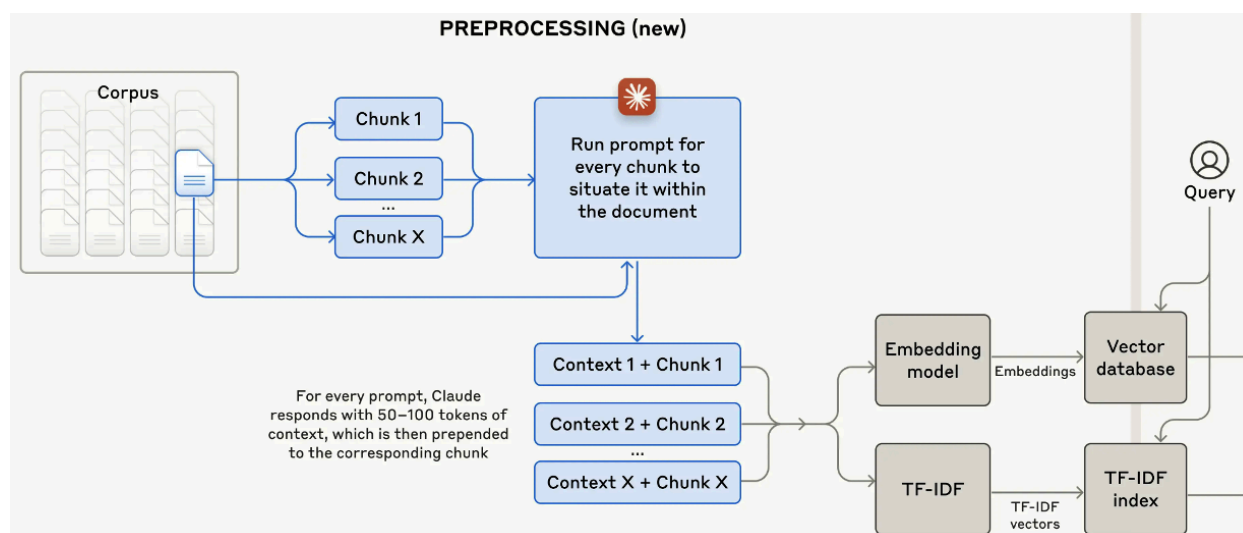
```
<document>
{{WHOLE_DOCUMENT}}
</document>
```

Here is the chunk we want to situate within the whole document:

```
<chunk>
{{CHUNK_CONTENT}}
</chunk>
```

Please give a short succinct context to situate this chunk within the overall document for the purposes of improving search retrieval of the chunk. Answer only with the succinct context and nothing else.

为每个块生成的上下文会添加到每个块的前面，然后将增强的块由检索算法索引。图6-5直观展示了Anthropic遵循的过程。



评估检索解决方案

在评估检索解决方案时，以下是需要记住的一些关键因素：

- 它支持哪些检索机制？是否支持混合搜索？
- 如果是向量数据库，它支持哪些嵌入模型和向量搜索算法？
- 它在数据存储和查询流量方面的可扩展性如何？它是否适用于你的流量模式？
- 索引数据需要多长时间？一次可以批量处理(添加/删除)多少数据？
- 不同检索算法的查询延迟是多少？
- 如果是托管解决方案，其定价结构是什么？是基于文档/向量数量还是基于查询数量？

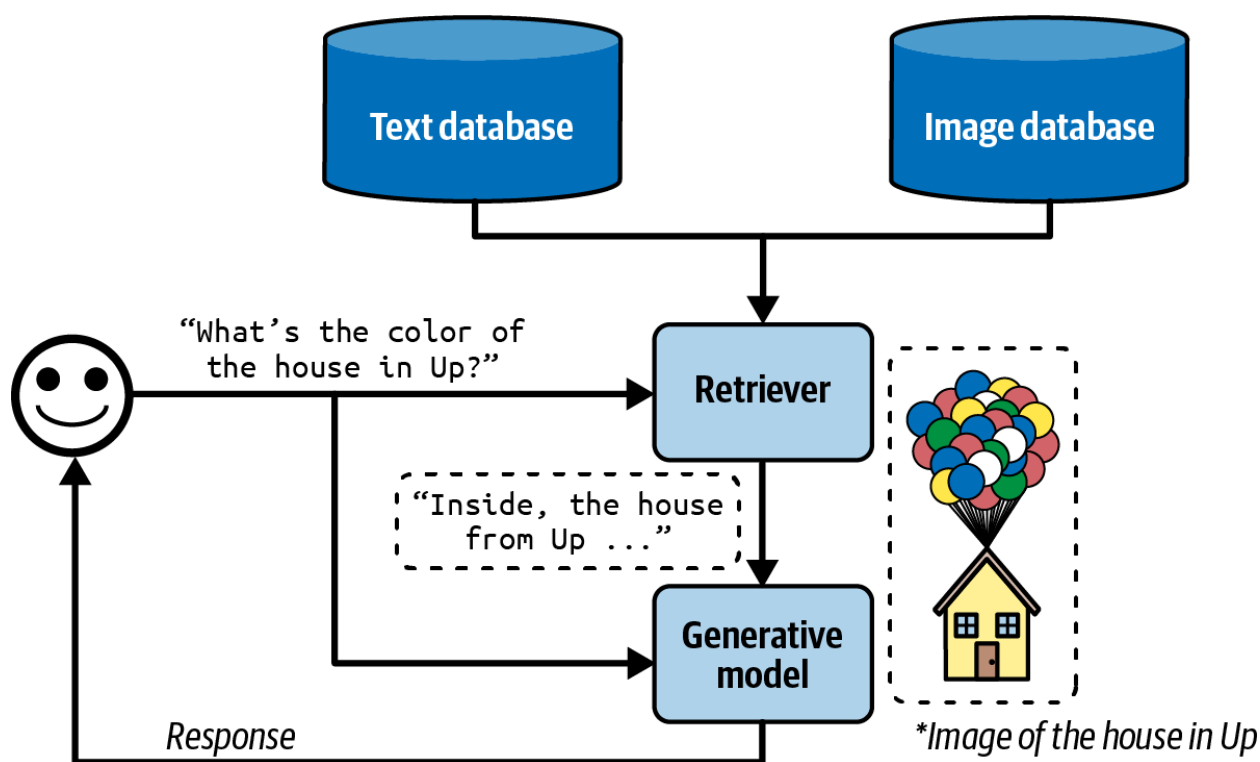
该列表不包括通常与企业解决方案相关的功能，如访问控制、合规性、数据平面和控制平面分离等。

RAG超越文本

上一节讨论了基于文本的RAG系统，其中外部数据源是文本文档。然而，外部数据源也可以是多模态和表格数据。

多模态RAG

如果你的生成器是多模态的，其上下文不仅可以用文本文档增强，还可以用图像、视频、音频等从外部源增强。为简洁起见，我将在示例中使用图像，但你可以用任何其他模态替换图像。给定查询，检索器同时获取与之相关的文本和图像。例如，给定“皮克斯电影《飞屋环游记》中房子的颜色是什么？”，检索器可以获取《飞屋环游记》中房子的图片来帮助模型回答，如图6-6所示。



如果图像有元数据——如标题、标签和说明——它们可以使用元数据检索。例如，如果图像的说明被认为与查询相关，则检索该图像。

如果你想基于图像内容检索图像，你需要有一种方法来比较图像和查询。如果查询是文本，你需要一个多模态嵌入模型，可以为图像和文本生成嵌入。假设你使用CLIP(Radford等，2021)作为多模态嵌入模型。检索器的工作方式如下：

1. 为所有数据(文本和图像)生成CLIP嵌入，并将它们存储在向量数据库中。
2. 给定查询，生成其CLIP嵌入。
3. 在向量数据库中查询所有嵌入接近查询嵌入的图像和文本。

带表格数据的RAG

大多数应用不仅使用非结构化数据如文本和图像，还使用表格数据。许多查询可能需要数据表中的信息来回答。使用表格数据增强上下文的工作流程与经典RAG工作流程有显著不同。

想象你为一个名为Kitty Vogue的电子商务网站工作，专门经营猫咪时尚。这家商店有一个名为Sales的订单表，如表6-3所示。

表6-3。虚构电子商务网站Kitty Vogue的订单表 (Sales) 示例

订单ID	时间戳	产品ID	产品	单价(\$)	数量	总计
1	...	2044	Meow Mix Seasoning	10.99	1	10.99
2	...	3492	Purr & Shake	25	2	50
3	...	2045	Fruity Fedora	18	1	18
...	...	...	...	...	...	...

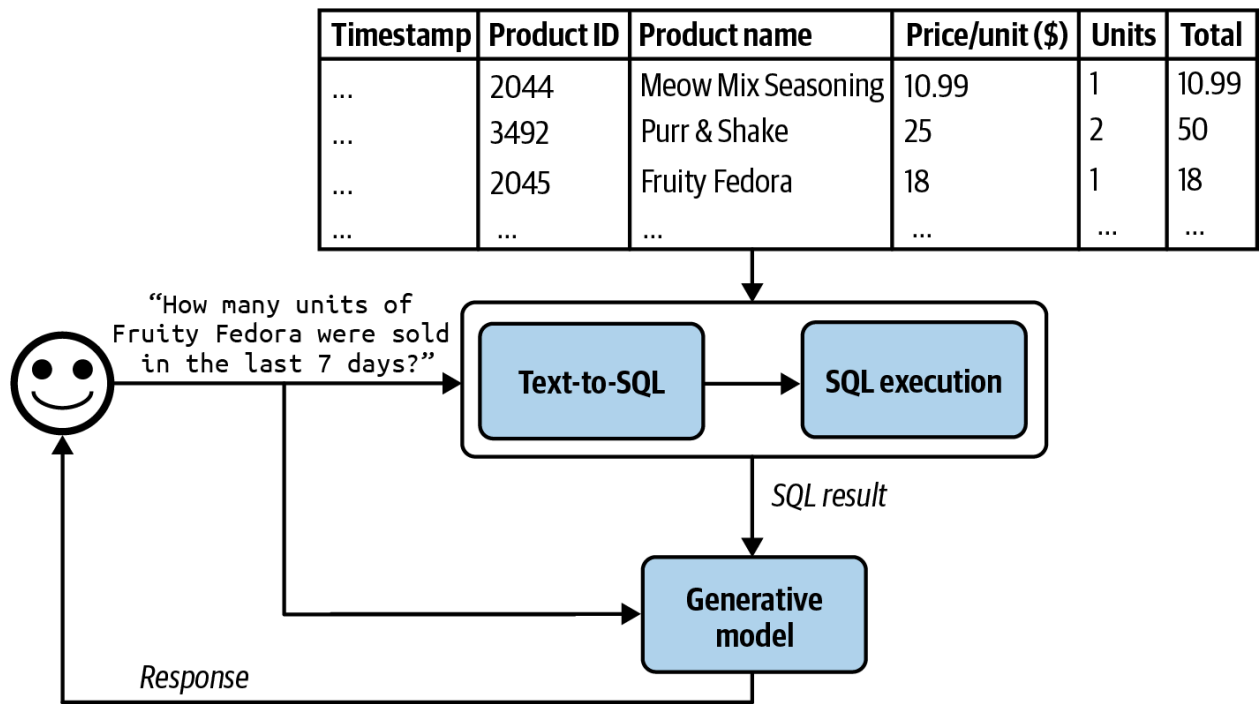
要生成对问题"过去7天里售出了多少件Fruity Fedora？"的响应，你的系统需要查询此表中所有涉及Fruity Fedora的订单，并汇总所有订单中的数量。假设可以使用SQL查询此表。SQL查询可能如下所示：

```
SELECT SUM(units) AS total_units_sold
FROM Sales
WHERE product_name = 'Fruity Fedora'
AND timestamp >= DATE_SUB(CURDATE(), INTERVAL 7 DAY);
```

工作流程如图6-7所示，如下：要运行此工作流程，你的系统必须能够生成并执行SQL查询：

- 1. 文本到SQL: 根据用户查询和提供的表模式，确定需要什么SQL查询。文本到SQL是语义解析的一个例子，如第2章所述。

- 2. SQL执行:执行SQL查询。
- 3. 生成:根据SQL结果和原始用户查询生成响应。



对于文本到SQL步骤，如果有许多可用表，其模式不能全部放入模型上下文中，你可能需要一个中间步骤来预测每个查询要使用哪些表。文本到SQL可以由生成最终响应的同一生成器完成，也可以由专门的文本到SQL模型完成。

在本节中，我们讨论了检索器和SQL执行器等工具如何使模型处理更多查询并生成更高质量的响应。给模型访问更多工具是否会进一步提高其能力？工具使用是智能体模式的核心特征，我们将在下一节讨论这一点。

智能体

智能体被许多人认为是AI的最终目标。Stuart Russell和Peter Norvig的经典著作《人工智能：现代方法》(Prentice Hall, 1995)将人工智能研究领域定义为“理性智能体的研究和设计”。

基础模型前所未有的能力为之前不可想象的智能体应用打开了大门。这些新能力使开发自主智能体作为我们的助手、同事和教练成为可能。它们可以帮助我们创建网站、收集数据、规划旅行、进行市场研究、管理客户账户、自动化数据输入、为面试做准备、面试候选人、谈判交易等。可能性似乎无穷无尽，这些智能体的潜在经济价值是巨大的。

警告
AI驱动的智能体是一个新兴领域，没有成熟的理论框架来定义、开发和评估它们。

本节是尝试从现有文献中构建框架的最佳努力，但它将随着领域的发展而演变。与本书的其余部分相比，本节更具实验性。

本节将从智能体概述开始，然后继续讨论决定智能体能力的两个方面：工具和规划。智能体凭借其新的操作模式，有新的失败模式。本节将以讨论如何评估智能体以捕捉这些失败结束。

尽管智能体是新颖的，但它们建立在本书中已经出现的概念之上，包括自我批评、思维链和结构化输出。

智能体概述

"智能体"一词在许多不同工程背景中使用，包括但不限于软件智能体、智能智能体、用户智能体、会话智能体和强化学习智能体。那么，什么是智能体？

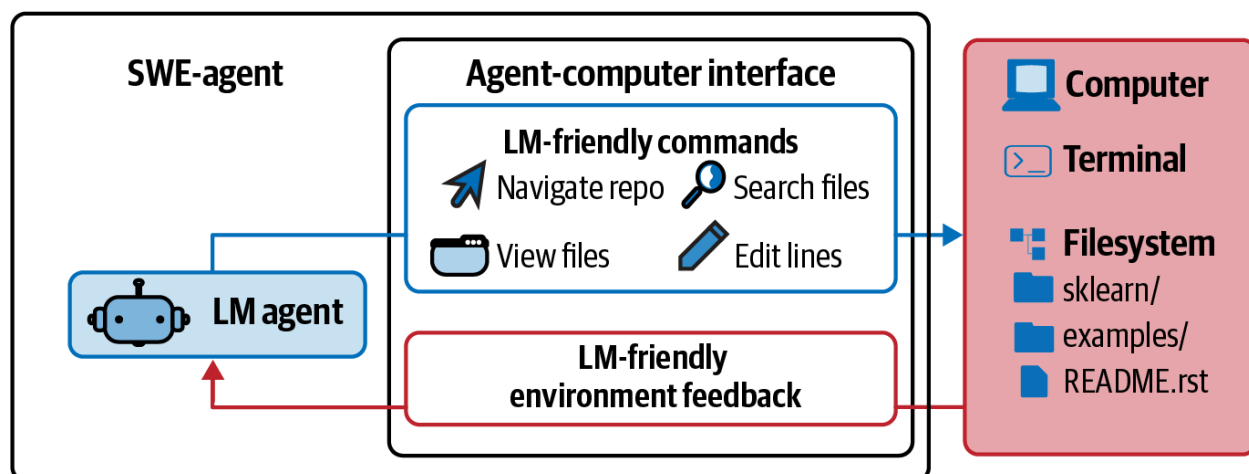
智能体是任何可以感知其环境并对该环境采取行动的东西。这意味着智能体的特征是它运行的环境和它可以执行的一组操作。

智能体可以运行的环境由其用例定义。如果智能体是为玩游戏(如Minecraft、围棋、Dota)开发的，那么该游戏就是其环境。如果你希望智能体从互联网抓取文档，环境就是互联网。如果你的智能体是烹饪机器人，厨房就是其环境。自动驾驶汽车智能体的环境是道路系统及其相邻区域。

AI智能体可以执行的操作集由它可以访问的工具增强。你每天交互的许多生成式AI驱动的应用程序实际上是可以访问工具的智能体，尽管是简单的。ChatGPT是一个智能体。它可以搜索网络、执行Python代码和生成图像。RAG系统是智能体，文本检索器、图像检索器和SQL执行器是它们的工具。

智能体的环境和其工具集之间存在强依赖关系。环境决定了智能体可能使用的工具。例如，如果环境是象棋游戏，智能体唯一可能的操作就是有效的象棋走法。然而，智能体的工具库限制了它可以运行的环境。例如，如果机器人唯一的动作是游泳，它将被限制在水环境中。

图6-8显示了SWE-agent(Yang等, 2024)的可视化，这是一个基于GPT-4的智能体。其环境是带有终端和文件系统的计算机。其操作集包括导航存储库、搜索文件、查看文件和编辑行。



AI智能体旨在完成通常由用户在输入中提供的任务。在AI智能体中，AI是大脑，处理接收到的信息，包括任务和环境反馈，规划一系列操作来实现这项任务，并确定任务是否已完成。

让我们回到Kitty Vogue示例中带表格数据的RAG系统。这是一个简单的智能体，有三个操作：响应生成、SQL查询生成和SQL查询执行。给定查询“预测未来三个月Fruity Fedora的销售收入”，智能体可能执行以下操作序列：

1. 推理如何完成此任务。它可能决定要预测未来销售，首先需要过去五年的销售数字。请注意，智能体的推理显示为其中间响应。
2. 调用SQL查询生成以生成查询，获取过去五年的销售数字。
3. 调用SQL查询执行以执行此查询。
4. 推理工具输出如何帮助销售预测。它可能决定这些数字不足以做出可靠预测，可能是因为缺少值。然后决定还需要关于过去营销活动的信息。
5. 调用SQL查询生成以生成过去营销活动的查询。
6. 调用SQL查询执行。
7. 推理这些新信息足以帮助预测未来销售。然后生成预测。
8. 推理任务已成功完成。

与非智能体用例相比，智能体通常需要更强大的模型，原因有二：

1. 复合错误：智能体通常需要执行多个步骤来完成任务，随着步骤数量增加，整体准确性会下降。如果模型每步准确率为95%，10个步骤后，准确率将降至60%，100个步骤后，准确率仅为0.6%。
2. 更高风险：由于可以访问工具，智能体能够执行更具影响力的任务，但任何失败可能导致更严重的后果。

需要许多步骤的任务可能需要时间和金钱来运行。然而，如果智能体能够自主，它们可以节省大量人力时间，使其成本值得。

给定环境，智能体在环境中的成功取决于它可以访问的工具库和其AI规划器的强度。让我们先了解模型可以使用的不同类型的工具。

工具

系统不需要访问外部工具就能成为智能体。然而，没有外部工具，智能体的能力将受到限制。独自一个模型通常只能执行一个操作——例如，LLM可以生成文本，图像生成器可以生成图像。外部工具使智能体更加强大。

工具帮助智能体感知环境并对其采取行动。允许智能体感知环境的操作是只读操作，而允许智能体对环境采取行动的操作是写入操作。

本节概述外部工具。如何使用工具将在"规划"部分讨论。

智能体可访问的工具集是其工具库。由于智能体的工具库决定了智能体能做什么，思考要给智能体哪些工具以及多少工具非常重要。更多工具赋予智能体更多能力。然而，工具越多，理解和有效利用它们就越具挑战性。如"工具选择"中所讨论的，需要实验来找到合适的工具集。

根据智能体的环境，可能有许多可能的工具。以下是你可能考虑三类工具：知识增强（即上下文构建）、能力扩展以及让智能体对其环境采取行动的工具。

知识增强

我希望本书到目前为止已经说服你了解到，拥有相关上下文对模型响应质量的重要性。一类重要的工具包括那些帮助增强智能体知识的工具。其中一些已经讨论过：文本检索器、图像检索器和SQL执行器。其他潜在的工具包括内部人员搜索、返回不同产品状态的库存API、Slack检索、电子邮件阅读器等。

许多这样的工具用你组织的私有流程和信息增强模型。然而，工具也可以让模型访问公共信息，特别是来自互联网的信息。

网络浏览是最早也是最受期待的被整合到ChatGPT等聊天机器人中的功能之一。网络浏览防止模型过时。当模型训练的数据变得过时，模型就会过时。如果模型的训练数据截止于上周，除非在上下文中提供相关信息，否则它将无法回答需要本周信息的问题。没有网络浏览，模型将无法告诉你天气、新闻、即将发生的事件、股票价格、航班状态等。

我使用"网络浏览"作为涵盖所有访问互联网的工具的总称，包括网络浏览器和特定API，如搜索API、新闻API、GitHub API或社交媒体API，如X、LinkedIn和Reddit的API。

虽然网络浏览允许你的智能体参考最新信息以生成更好的响应并减少幻觉，但它也可能使你的智能体暴露在互联网的污秽环境中。谨慎选择你的互联网API。

能力扩展

要考虑的第二类工具是那些解决AI模型固有限制的工具。它们是给模型性能提升的简单方法。例如，AI模型以不擅长数学而闻名。如果你问模型199,999除以292是多少，模型很可能会失败。然而，如果模型可以访问计算器，这个计算就变得很简单。与其尝试训练模型擅长算术，提供模型访问工具的权限更为资源高效。

其他可以显著提升模型能力的简单工具包括日历、时区转换器、单位转换器（例如，从磅到千克）以及可以翻译模型不擅长的语言的翻译器。

更复杂但功能强大的工具是代码解释器。与其训练模型理解代码，你可以给它访问代码解释器的权限，使其能够执行代码片段，返回结果或分析代码失败原因。这种能力让你的智能体可以充当编码助手、数据分析师，甚至是可以编写代码运行实验并报告结果的研究助手。然而，自动代码执行带来代码注入攻击的风险，如“防御性提示工程”中所讨论的。适当的安全措施对保障你和你的用户安全至关重要。

外部工具可以使仅文本或仅图像的模型变为多模态。例如，只能生成文本的模型可以利用文本到图像模型作为工具，使其能够同时生成文本和图像。给定文本请求，智能体的AI规划器决定是调用文本生成、图像生成还是两者都调用。这就是ChatGPT能够同时生成文本和图像的方式——它使用DALL-E作为图像生成器。智能体还可以使用代码解释器生成图表和图形，使用LaTeX编译器渲染数学方程，或使用浏览器从HTML代码渲染网页。

类似地，只能处理文本输入的模型可以使用图像说明工具处理图像，使用转录工具处理音频。它可以使用OCR（光学字符识别）工具阅读PDF。

工具使用可以显著提升模型的性能，相比仅使用提示甚至微调。Chameleon(Lu等, 2023)显示，由GPT-4驱动的智能体，增强了13种工具，可以在几个基准测试上超越单独的GPT-4。这个智能体使用的工具示例包括知识检索、查询生成器、图像说明器、文本检测器和Bing搜索。

在ScienceQA上，一个科学问答基准，Chameleon将最佳已发布的少样本结果提高了11.37%。在TabMWP（表格数学文字问题）(Lu等, 2022)上，一个涉及表格数学问题的基准，Chameleon将准确率提高了17%。

写入操作

到目前为止，我们讨论了允许模型从数据源读取的只读操作。但工具也可以执行写入操作，对数据源进行更改。SQL执行器可以检索数据表（读取），但也可以更改或删除表（写入）。电子邮件API可以读取电子邮件，但也可以回复它。银行API可以检索你当前的余额，但也可以发起银行转账。

写入操作使系统能够做更多事情。它们可以使你自动化整个客户外联工作流程：研究潜在客户、查找他们的联系方式、起草电子邮件、发送首封电子邮件、阅读回复、跟进、提取订单、用新订单更新数据库等。

然而，赋予AI自动改变我们生活的能力是令人恐惧的。正如你不应该给实习生删除生产数据库的权限，你也不应该允许不可靠的AI发起银行转账。对系统能力和安全措施的信心至关重要。你需要确保系统受到保护，防止不良行为者试图操纵它执行有害行动。

当我向一群人谈论自主AI智能体时，通常有人会提到自动驾驶汽车。"如果有人黑入汽车绑架你怎么办？"虽然自动驾驶汽车的例子因其物理性而看起来很实际，但AI系统无需在物理世界中存在就能造成伤害。它可以操纵股市、窃取版权、侵犯隐私、强化偏见、传播错误信息和宣传等，如"防御性提示工程"中所讨论的。

这些都是有效的担忧，任何希望利用AI的组织都需要认真对待安全和安保。然而，这并不意味着AI系统永远不应该被赋予在现实世界中行动的能力。如果我们能让人们信任机器将我们送入太空，我希望有一天，安全措施将足以让我们信任自主AI系统。此外，人类也会失败。就个人而言，我会更信任自动驾驶汽车比普通陌生人更能安全地驾驶我。

就像正确的工具可以帮助人类大大提高生产力——你能想象没有Excel做生意或没有起重机建造摩天大楼吗？——工具使模型能够完成更多任务。许多模型提供商已经支持他们的模型使用工具，这一功能通常称为函数调用。展望未来，我预计函数调用与大多数模型的广泛工具集将成为常见。

规划

基础模型智能体的核心是负责解决任务的模型。任务由其目标和约束定义。例如，一个任务是安排从旧金山到印度的两周旅行，预算为5,000美元。目标是两周旅行。约束是预算。

复杂任务需要规划。规划过程的输出是计划，即概述完成任务所需步骤的路线图。有效规划通常要求模型理解任务，考虑实现此任务的不同选项，并选择最有希望的一个。

如果你曾参加过任何规划会议，你就知道规划很难。作为一个重要的计算问题，规划已经得到充分研究，需要几卷书才能涵盖。我在这里只能涵盖表面内容。

规划概述

给定任务，有许多可能的分解方式，但不是所有方式都会带来成功结果。在正确的解决方案中，有些比其他更有效。考虑查询"有多少没有收入的公司筹集了至少10亿美元？"有许多可能的解决方法，但作为说明，考虑两个选项：

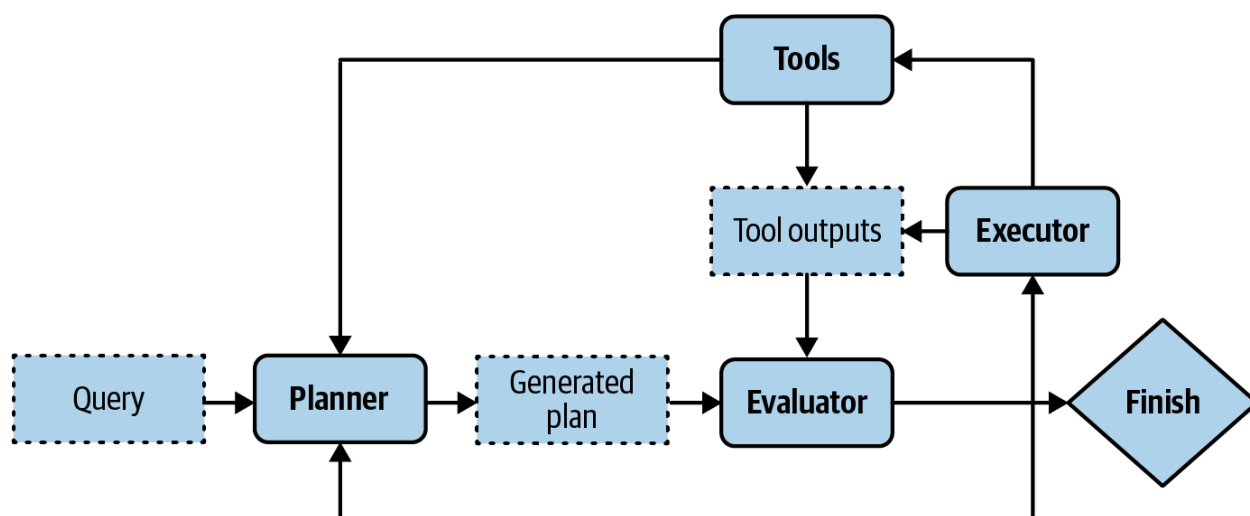
1. 找出所有没有收入的公司，然后按筹集金额过滤。
2. 找出所有筹集至少10亿美元的公司，然后按收入过滤。

第二个选项更有效。没有收入的公司比筹集至少10亿美元的公司多得多。只考虑这两个选项，智能智能体应该选择选项2。

你可以在同一提示中将规划与执行结合起来。例如，你给模型一个提示，要求它一步步思考(例如使用思维链提示)，然后在一个提示中执行这些步骤。但如果模型提出一个1,000步的计划，甚至无法完成目标怎么办？如果没有监督，智能体可能会运行这些步骤数小时，在API调用上浪费时间和金钱，然后你才意识到它没有取得进展。

为避免无用的执行，规划应与执行分离。你首先要求智能体生成计划，只有在验证此计划后才执行它。计划可以使用启发式方法验证。例如，一个简单的启发式方法是消除包含无效操作的计划。如果生成的计划需要Google搜索，而智能体无法访问Google搜索，则此计划无效。另一个简单的启发式方法可能是消除所有超过X步的计划。计划也可以使用AI评判进行验证。你可以要求模型评估计划是否合理或如何改进它。

如果生成的计划被评估为不好，你可以要求规划器生成另一个计划。如果生成的计划很好，则执行它。如果计划包含外部工具，将调用函数调用。执行此计划的输出将再次需要评估。请注意，生成的计划不必是整个任务的端到端计划。它可以是子任务的小计划。整个过程如图6-9所示。



你的系统现在有三个组件：一个生成计划，一个验证计划，另一个执行计划。如果你将每个组件视为一个智能体，这就是一个多智能体系统。

为加快过程，你可以并行生成几个计划，而不是按顺序生成，并要求评估者选择最有希望的一个。这是另一个延迟/成本权衡，因为同时生成多个计划将产生额外成本。

规划需要理解任务背后的意图：用户想用这个查询做什么？意图分类器通常用于帮助智能体规划。如“将复杂任务分解为更简单的子任务”所示，意图分类可以使用另一个提示或为此任务训练的分类模型完成。意图分类机制可以被视为你多智能体系统中的另一个智能体。

了解意图可以帮助智能体选择正确的工具。例如，对于客户支持，如果查询是关于账单，智能体可能需要访问检索用户最近付款的工具。但如果查询是关于如何重置密码，智能体可能需要访问文档检索。

提示

一些查询可能超出智能体的范围。意图分类器应能将请求分类为“不相关”，使智能体可以礼貌拒绝，而不是浪费FLOP提出不可能的解决方案。

到目前为止，我们假设智能体自动化所有三个阶段：生成计划、验证计划和执行计划。在现实中，人类可以参与任何阶段帮助过程并减轻风险。人类专家可以提供计划、验证计划或执行计划的部分。例如，对于智能体难以生成整个计划的复杂任务，人类专家可以提供高级计划，智能体可以在此基础上扩展。如果计划涉及风险操作，如更新数据库或合并代码更改，系统可以在执行前请求明确的人类批准，或让人类执行这些操作。为使这成为可能，你需要明确定义智能体对每个操作可以拥有的自动化级别。

总结一下，解决任务通常涉及以下过程。请注意，反思对智能体不是强制性的，但它会显著提升智能体的性能：

1. 计划生成：为完成此任务制定计划。计划是一系列可管理的操作，所以这个过程也称为任务分解。
2. 反思和错误纠正：评估生成的计划。如果是不好的计划，生成新计划。
3. 执行：执行生成计划中概述的操作。这通常涉及调用特定函数。
4. 反思和错误纠正：收到操作结果后，评估这些结果并确定目标是否已完成。识别并纠正错误。如果目标未完成，生成新计划。

你已经在本书中看到了一些计划生成和反思的技术。当你要求模型“一步步思考”时，你是在要求它分解任务。当你要求模型“验证你的答案是否正确”时，你是在要求它反思。

基础模型作为规划器

一个开放问题是基础模型能规划得多好。许多研究者认为基础模型，至少那些建立在自回归语言模型之上的模型，不能。Meta的首席AI科学家Yann LeCun明确表示自回归LLM不能规划(2023)。在“LLM真的能推理和规划吗？”一文中，Kambhampati(2023)认为LLM擅长提取知识但不擅长规划。Kambhampati认为，声称LLM具有规划能力的论文混淆了从LLM提取的一般规划知识与可执行计划。“从LLM产生的计划对普通用户可能看起来合理，但可能导致执行时的交互和错误。”

然而，虽然有很多关于LLM是糟糕规划器的轶事证据，目前尚不清楚这是因为我们不知道如何正确使用LLM，还是因为LLM从根本上不能规划。

规划，从本质上讲，是一个搜索问题。你在不同通往目标的路径中搜索，预测每条路径的结果(奖励)，并选择最有希望的路径。通常，你可能会确定不存在可以带你到目标的路径。

搜索通常需要回溯。例如，想象你处于有两个可能操作的步骤：A和B。采取操作A后，你进入一个不太有希望的状态，因此你需要回溯到前一状态采取操作B。

有些人认为自回归模型只能生成前向操作。它不能回溯生成替代操作。因此，他们得出结论，自回归模型不能规划。然而，这不一定是真的。在执行操作A的路径后，如果模型确定此路径没有意义，它可以修改路径改用操作B，有效地回溯。模型也可以始终重新开始并选择另一条路径。

也有可能LLM是糟糕的规划器，因为它们没有获得规划所需的工具。要规划，不仅需要知道可用的操作，还需要知道每个操作的潜在结果。作为一个简单例子，假设你想爬山。你的潜在操作是向

右转、向左转、转身或直走。然而，如果向右转会使你掉下悬崖，你可能不想考虑这个操作。在技术术语中，操作使你从一个状态转移到另一个状态，需要知道结果状态才能决定是否采取操作。

这意味着仅提示模型生成操作序列如流行的思维链提示技术所做的那样是不够的。论文"使用语言模型推理是用世界模型规划"(Hao等, 2023)认为，LLM包含如此多关于世界的信息，能够预测每个操作的结果。这个LLM可以结合这种结果预测生成连贯的计划。

即使AI不能规划，它仍然可以是规划器的一部分。可能可以用搜索工具和状态跟踪系统增强LLM来帮助它规划。

基础模型(FM)与强化学习(RL)规划器

智能体是RL的核心概念，RL在维基百科中定义为"关注智能智能体如何在动态环境中采取行动以最大化累积奖励的领域"。

RL智能体和FM智能体在许多方面相似。它们都以环境和可能的操作为特征。主要区别在于它们的规划器如何工作。在RL智能体中，规划器由RL算法训练。训练这个RL规划器可能需要大量时间和资源。在FM智能体中，模型就是规划器。可以提示或微调此模型以提高其规划能力，通常需要更少的时间和资源。

然而，没有什么能阻止FM智能体结合RL算法来提高其性能。我怀疑长期来看，FM智能体和RL智能体将合并。

计划生成

将模型转变为计划生成器最简单的方法是通过提示工程。想象你想创建一个智能体帮助客户了解Kitty Vogue的产品。你给这个智能体访问三个外部工具的权限：按价格检索产品、检索顶级产品和检索产品信息。以下是计划生成的一个示例提示。这个提示仅供说明，生产提示可能更复杂：

SYSTEM PROMPT

Propose a plan to solve the task. You have access to 5 actions:

`get_today_date()`

`fetch_top_products(start_date, end_date, num_products)`

`fetch_product_info(product_name)`

`generate_query(task_history, tool_output)`

`generate_response(query)`

The plan must be a sequence of valid actions.

Examples

Task: "Tell me about Fruity Fedora"

Plan: `[fetch_product_info, generate_query, generate_response]`

Task: "What was the best selling product last week?"

Plan: `[fetch_top_products, generate_query, generate_response]`

Task: {USER INPUT}
Plan:

关于这个例子有两点需要注意：

1. 这里使用的计划格式——一个函数列表，其参数由智能体推断——只是构建智能体控制流的众多方式之一。
2. generate_query函数输入当前任务历史和最近的工具输出，生成将输入到响应生成器的查询。每一步的工具输出都会添加到任务历史中。

给定用户输入"上周销售最好的产品的价格是多少"，生成的计划可能如下所示：

```
1. get_time()
2. fetch_top_products()
3. fetch_product_info()
4. generate_query()
5. generate_response()
```

你可能想，"每个函数所需的参数怎么办？"由于参数通常从之前的工具输出中提取，因此很难提前预测确切的参数。如果第一步get_time()输出"2030-09-13"，那么智能体可以推理下一步应该使用以下参数调用：

```
retrieve_top_products(
    start_date="2030-09-07",
    end_date="2030-09-13",
    num_products=1
)
```

通常，没有足够的信息来确定函数的确切参数值。例如，如果用户问，"畅销产品的平均价格是多少？"，以下问题的答案不明确：

- 用户想查看多少畅销产品？
- 用户想查看上周、上月还是所有时间的畅销产品？

这意味着模型经常不得不猜测，而猜测可能是错误的。

由于行动序列和相关参数都由AI模型生成，它们可能是幻觉。幻觉可能导致模型调用无效函数或调用有效函数但使用错误参数。提高模型一般性能的技术可用于提高模型的规划能力。

以下是几种让智能体更善于规划的方法：

- 编写更好的系统提示，提供更多例子。
- 提供工具及其参数的更好描述，使模型更好地理解它们。
- 重写函数本身，使其更简单，例如将复杂函数重构为两个更简单的函数。
- 使用更强大的模型。一般来说，更强大的模型更擅长规划。
- 微调模型进行计划生成。

函数调用

许多模型提供商为他们的模型提供工具使用，有效地将模型转变为智能体。工具就是函数。因此，调用工具通常称为函数调用。不同的模型API工作方式不同，但通常，函数调用的工作方式如下：

1. 创建工具库。
 - 声明你可能希望模型使用的所有工具。每个工具都由其执行入口点（例如，其函数名称）、其参数及其文档（例如，该函数做什么以及需要什么参数）描述。
2. 指定智能体可以使用的工具。
 - 由于不同查询可能需要不同工具，许多API允许你指定每个查询使用的声明工具列表。有些让你通过以下设置进一步控制工具使用：
 - `required`: 模型必须使用至少一个工具。
 - `none`: 模型不应使用任何工具。
 - `auto`: 模型决定使用哪些工具。

函数调用如图6-10所示。这是用伪代码编写的，以使其代表多个API。要使用特定API，请参阅其文档。

```
def lbs_to_kg(lbs):
    return lbs * 0.45359237
```

```
def ft_to_meters(ft):
    return ft * 0.3048
```

Tool definition

```
lbs_to_kg_tool = FunctionDeclaration(
    name="lbs_to_kg",
    description="Convert from pounds to kilograms",
    parameters={
        "type": "object",
        "properties": {
            "lbs": {"type": "int", "description": "The value to be converted"}
        },
        "required": ["lbs"]
    },
)
```

(1) Tool description

```
ft_to_m_tool = FunctionDeclaration(name="ft_to_meters",...)
```

```
messages = [{"role": "user", "content": [USER_QUERY]}]
```

User query

```
response = model_client.chat.completions.create(
    model=[MODEL_NAME],
    messages=messages,
    tools=[lbs_to_kg_tool, ft_to_m_tool],
    tool_choice="auto",
)
```

(2) This query can use two tools

给定查询，如图6-10所定义的智能体将自动生成要使用的工具及其参数。有些函数调用API将确保只生成有效函数，尽管它们无法保证正确的参数值。

例如，给定用户查询“40磅等于多少千克？”，智能体可能决定它需要lbs_to_kg_tool工具，参数值为40。智能体的响应可能如下所示：

```
response = ModelResponse(
    finish_reason='tool_calls',
    message=chat.Message(
        content=None,
        role='assistant',
        tool_calls=[
            ToolCall(
                function=Function(
                    arguments='{"lbs":40}',
                    name='lbs_to_kg'),
                type='function')
        ]
    )
)
```

从此响应中，你可以调用函数lbs_to_kg(lbs=40)并使用其输出生成对用户的响应。

提示

使用智能体时，始终要求系统报告它为每个函数调用使用的参数值。检查这些值以确保它们是正确的。

规划粒度

计划是概述完成任务所需步骤的路线图。路线图可以有不同粒度级别。要规划一年，按季度计划比按月计划更高级，而按月计划又比按周计划更高级。

存在规划/执行权衡。详细计划更难生成但更容易执行。高级计划更容易生成但更难执行。避免这种权衡的一种方法是分层规划。首先，使用规划器生成高级计划，如按季度计划。然后，对于每个季度，使用相同或不同的规划器生成按月计划。

到目前为止，所有生成计划的例子都使用确切的函数名称，这非常精细。这种方法的问题是智能体的工具库可能随时间变化。例如，获取当前日期的函数get_time()可能重命名为get_current_time()。当工具更改时，你需要更新提示和所有示例。使用确切的函数名称也使得在具有不同工具API的不同用例之间重用规划器更加困难。

如果你之前已经微调了模型基于旧工具库生成计划，你需要在新工具库上再次微调模型。

为避免这个问题，计划也可以使用更自然的语言生成，这比特定领域的函数名称更高级。例如，给定查询“上周销售最好的产品的价格是多少”，可以指示智能体输出如下计划：

1. get current date
2. retrieve the best-selling product last week
3. retrieve product information
4. generate query
5. generate response

使用更自然的语言有助于使你的计划生成器对工具API的变化更加健壮。如果你的模型主要在自然语言上训练，它可能更善于理解和生成自然语言计划，并且更不可能产生幻觉。

这种方法的缺点是你需要一个翻译器将每个自然语言操作翻译成可执行命令。然而，翻译是一个比规划简单得多的任务，可以由更弱的模型完成，幻觉风险更低。

复杂计划

到目前为止的计划示例都是顺序的：计划中的下一个操作总是在前一个操作完成后执行。操作可以执行的顺序称为控制流。顺序形式只是控制流的一种类型。其他类型的控制流包括并行、if语句和for循环。以下列表概述了每种控制流，包括顺序以供比较：

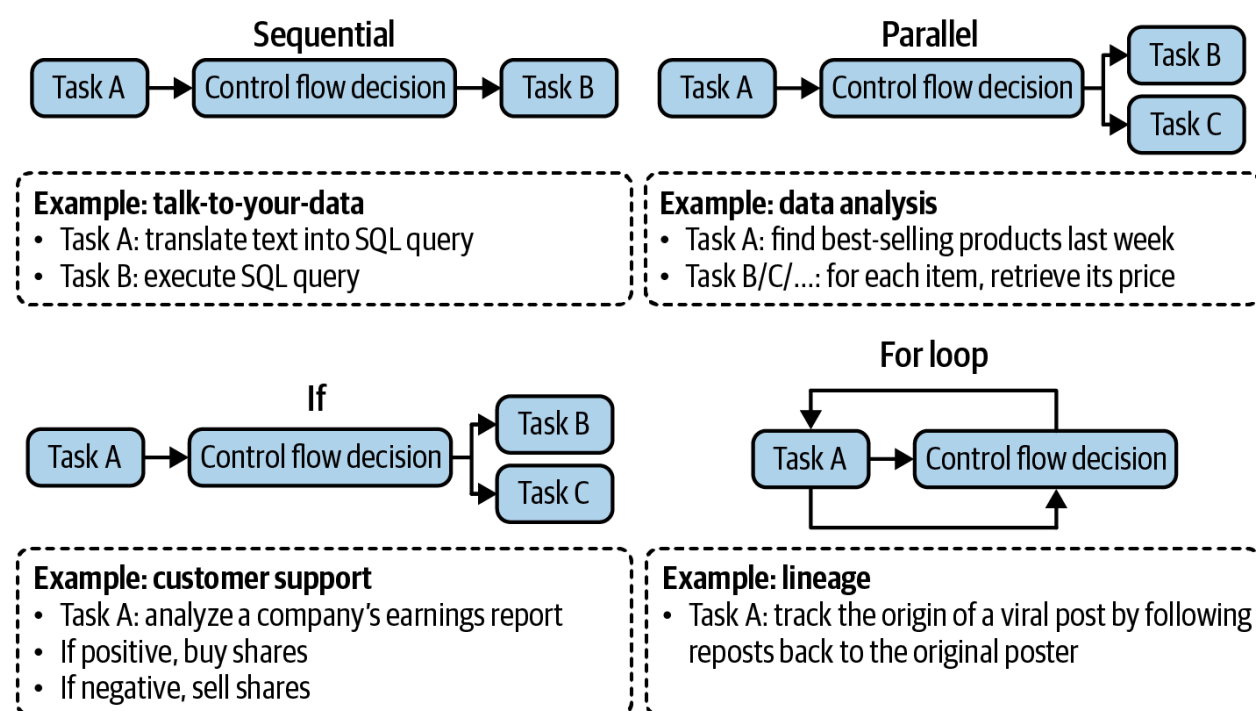
顺序 在任务A完成后执行任务B，可能是因为任务B依赖于任务A。例如，只有在从自然语言输入翻译后执行SQL查询之后才能执行。

并行 同时执行任务A和B。例如，给定查询"找到售价低于100美元的畅销产品"，智能体可能首先检索前100个畅销产品，然后对于这些产品中的每一个，检索其价格。

If语句 根据前一步骤的输出执行任务B或任务C。例如，智能体首先检查NVIDIA的盈利报告。基于这份报告，它可以决定卖出或买入NVIDIA股票。

For循环 重复执行任务A直到满足特定条件。例如，不断生成随机数直到得到一个质数。

这些不同的控制流程如图6-11所示。



在传统软件工程中，控制流的条件是精确的。在AI驱动的智能体中，AI模型决定控制流。具有非顺序控制流的计划更难生成，也更难翻译成可执行命令。

评估智能体框架时，检查它支持哪些控制流。例如，如果系统需要浏览十个网站，它能同时进行吗？并行执行可以显著减少用户感知的延迟。

反思和错误纠正

即使是最好的计划也需要不断评估和调整，以最大化成功的机会。虽然反思对智能体的运行不是绝对必要的，但对智能体的成功是必要的。

反思在任务过程中的多个地方很有用：

- 收到用户查询后，评估请求是否可行。
- 初始计划生成后，评估计划是否有意义。
- 每个执行步骤后，评估是否在正确轨道上。
- 整个计划执行后，确定任务是否已完成。

反思和错误纠正是携手并进的两种不同机制。反思产生见解，帮助发现需要纠正的错误。

反思可以使用相同的智能体通过自我批评提示完成。它也可以使用单独的组件完成，例如专门的评分器：一个为每个结果输出具体分数的模型。

ReAct(Yao等, 2022)首次提出，交替推理和行动已成为智能体的常见模式。Yao等人使用术语"推理"包括规划和反思。在每一步，要求智能体解释其思考(规划)，采取行动，然后分析观察结果(反思)，直到智能体认为任务已完成。通常通过示例提示智能体以以下格式生成输出：

```
Thought 1: ...
Act 1: ...
Observation 1: ...

... [continue until reflection determines that the task is finished] ...

Thought N: ...
Act N: Finish [Response to query]
```

图6-12显示了一个遵循ReAct框架的智能体响应HotpotQA(Yang等, 2018)中的问题的示例，这是一个多跳问答的基准。

你可以在多智能体设置中实现反思：一个智能体规划并采取行动，另一个智能体在每一步或几步后评估结果。

如果智能体的响应未能完成任务，你可以提示智能体反思为什么失败以及如何改进。基于这个建议，智能体生成一个新计划。这使智能体能够从错误中学习。例如，给定一个代码生成任务，评估者可能评估生成的代码在1/3的测试用例中失败。智能体然后反思失败的原因是因为它没有考虑所有数字都是负数的数组。然后，执行者生成新代码，考虑全负数数组。

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: `Search[Apple Remote]`

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: `Search[Front Row]`

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: `Search[Front Row (software)]`

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

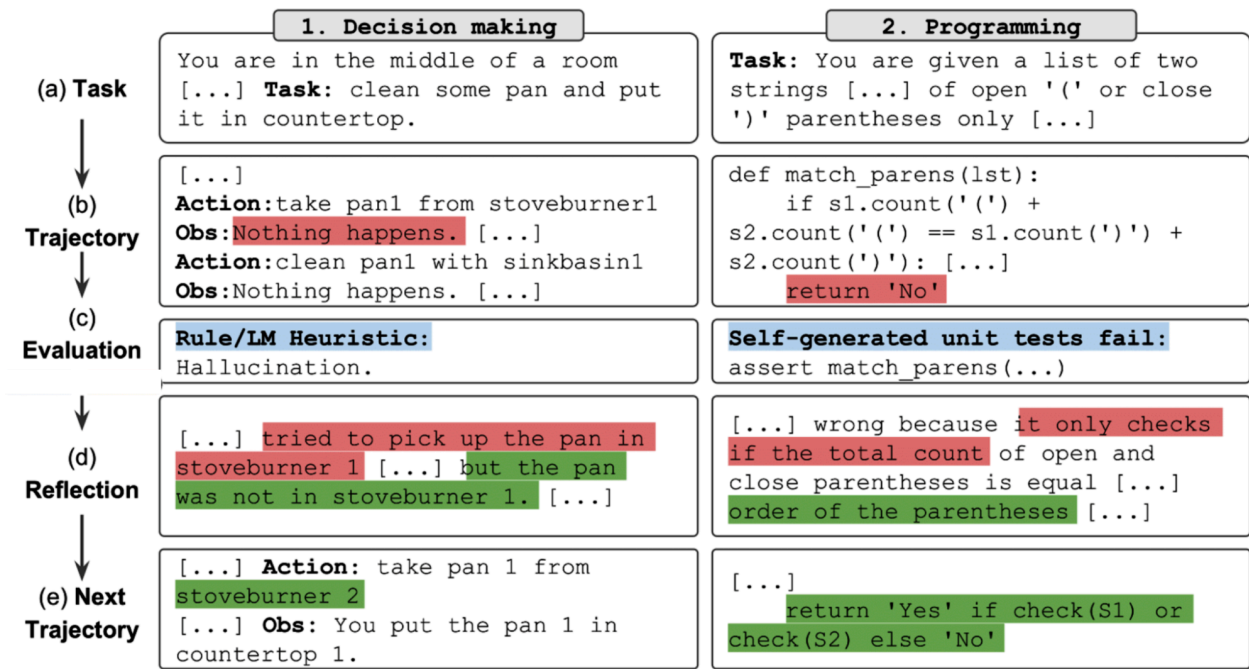
Act 4: `Finish[keyboard function keys]`



这是Reflexion(Shinn等, 2023)采取的方法。在这个框架中, 反思分为两个模块: 一个评估结果的评估器和一个分析出了什么问题的自我反思模块。图6-13显示了Reflexion智能体在行动中的例子。作者使用术语"轨迹"来指代计划。在每一步, 在评估和自我反思之后, 智能体提出一个新轨迹。

与计划生成相比, 反思相对容易实现, 并且可以带来令人惊讶的良好性能改进。这种方法的缺点是延迟和成本。思考、观察, 有时候行动可能需要很多令牌来生成, 这增加了成本和用户感知的延迟, 特别是对于有许多中间步骤的任务。为了促使他们的智能体遵循格式, ReAct和Reflexion作

者在提示中使用了大量示例。这增加了计算输入令牌的成本，并减少了可用于其他信息的上下文空间。



工具选择

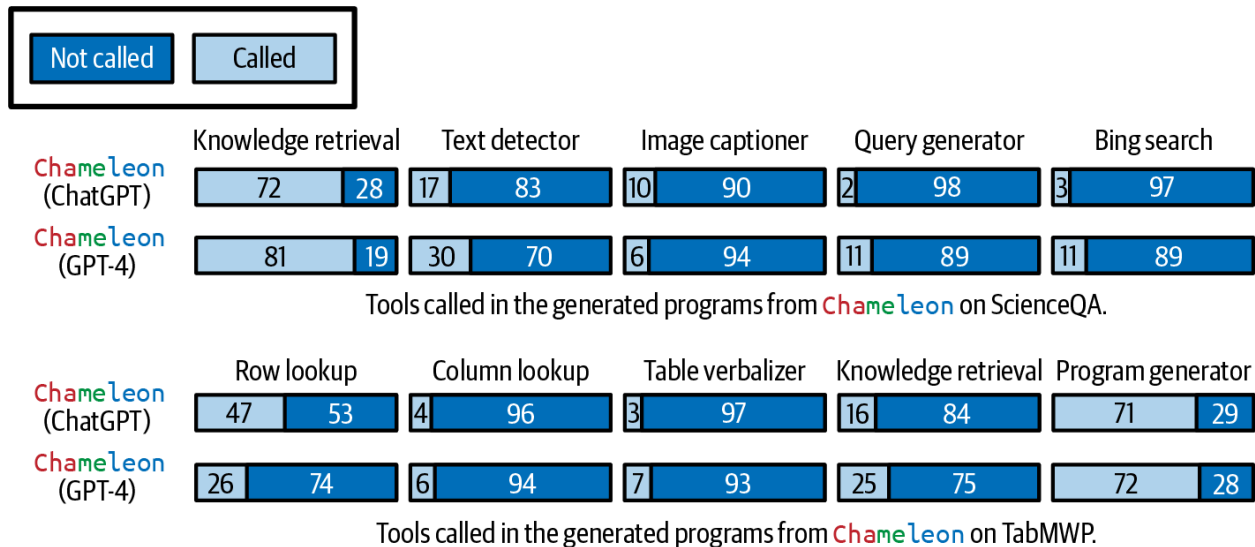
因为工具通常在任务成功中扮演关键角色，工具选择需要仔细考虑。给智能体的工具取决于环境和任务，但也取决于驱动智能体的AI模型。

没有关于如何选择最佳工具集的万无一失的指南。智能体文献包含各种各样的工具库。例如，Toolformer(Schick等, 2023)微调GPT-J学习了五种工具。Chameleon(Lu等, 2023)使用13种工具。另一方面，Gorilla(Patil等, 2023)尝试提示智能体在1,645个API中选择正确的API调用。

更多工具给智能体更多能力。然而，工具越多，有效使用它们就越困难。这类似于人类更难掌握大量工具。添加工具也意味着增加工具描述，这可能不适合模型的上下文。

像构建AI应用程序时的许多其他决策一样，工具选择需要实验和分析。以下是一些可以帮助你决定的事情：

- 比较智能体使用不同工具集的表现。
- 进行消融研究，看看如果从工具库中移除一个工具，智能体的性能会下降多少。如果可以在不降低性能的情况下移除工具，就移除它。
- 寻找智能体经常犯错的工具。如果一个工具对智能体来说太难使用——例如，大量提示甚至微调都不能让模型学会使用它——就改变工具。
- 绘制工具调用分布图，看看哪些工具最常用，哪些工具最少用。图6-14显示了Chameleon(Lu等, 2023)中GPT-4和ChatGPT工具使用模式的差异。



Lu等(2023)的实验也证明了两点：

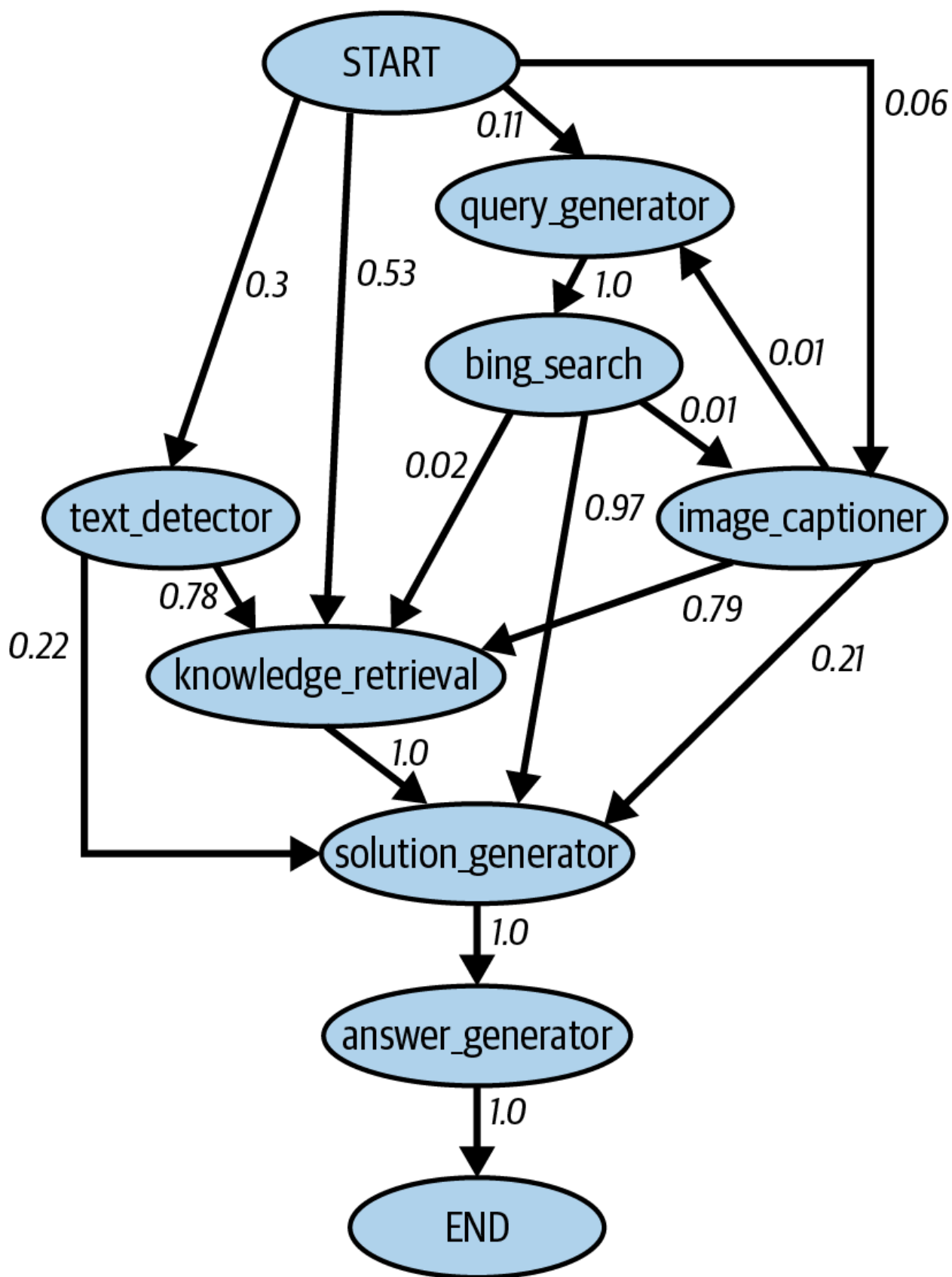
- 1. 不同任务需要不同工具。ScienceQA，科学问答任务，比TabMWP，表格数学问题解决任务，更依赖知识检索工具。
- 2. 不同模型有不同的工具偏好。例如，GPT-4似乎选择比ChatGPT更广泛的工具集。ChatGPT似乎偏爱图像说明，而GPT-4似乎偏爱知识检索。

提示
评估智能体框架时，评估它支持哪些规划器和工具。不同框架可能专注于不同类别的工具。例如，AutoGPT专注于社交媒体API(Reddit、X和维基百科)，而Composio专注于企业API(Google Apps、GitHub和Slack)。

由于你的需求可能随时间变化，评估扩展你的智能体以纳入新工具的难易程度。

作为人类，我们变得更有生产力不仅是通过使用给予我们的工具，还通过从更简单的工具创造越来越强大的工具。AI能否从其初始工具创建新工具？

Chameleon(Lu等，2023)提出了工具转换的研究：在工具X之后，智能体调用工具Y的可能性有多大？图6-15显示了工具转换的例子。如果两个工具经常一起使用，它们可以被组合成一个更大的工具。如果智能体意识到这些信息，智能体本身可以组合初始工具，不断构建更复杂的工具。



Vogager(Wang等, 2023)提出了一个技能管理器, 跟踪智能体获得的新技能(工具)以便以后重用。每个技能都是一个编码程序。当技能管理器确定新创建的技能有用(例如, 因为它成功帮助智能体完成任务)时, 它将此技能添加到技能库(在概念上类似于工具库)。以后可以检索此技能用于其他任务。

在本节前面, 我们提到智能体在环境中的成功取决于其工具库和规划能力。任一方面的失败都可能导致智能体失败。下一节将讨论智能体的不同失败模式以及如何评估它们。

智能体失败模式和评估

评估是关于检测失败。智能体执行的任务越复杂, 可能的失败点就越多。除了第3章和第4章中讨论的所有AI应用程序常见失败模式外, 智能体还有由规划、工具执行和效率引起的独特失败。有些失败比其他更容易捕捉。

要评估智能体, 确定其失败模式并测量每种失败模式发生的频率。

我创建了一个简单的基准来说明这些不同的失败模式, 你可以在本书的GitHub存储库中看到。还有其他智能体基准和排行榜, 如Berkeley函数调用排行榜、AgentOps评估工具和TravelPlanner基准。

规划失败

规划很难, 可能以多种方式失败。最常见的规划失败模式是工具使用失败。智能体可能生成具有以下一个或多个错误的计划:

无效工具

例如, 它生成包含bing_search的计划, 但bing_search不在智能体的工具库中。

有效工具, 无效参数

例如, 它调用lbs_to_kg带两个参数。lbs_to_kg在工具库中但只需要一个参数, lbs。

有效工具, 不正确的参数值

例如, 它调用lbs_to_kg带一个参数lbs, 但使用100作为lbs的值, 而应该是120。

另一种规划失败模式是目标失败: 智能体未能实现目标。这可能是由于计划没有解决任务, 或者解决任务但没有遵循约束。为了说明这一点, 想象你要求模型规划从旧金山到河内的两周旅行, 预算5,000美元。智能体可能规划从旧金山到胡志明市的旅行, 或者规划从旧金山到河内的两周旅行, 但预算远超。

智能体评估经常忽视的一个常见约束是时间。在许多情况下, 智能体花费的时间关系不大, 因为你可以分配任务给智能体, 只需在完成时检查。然而, 在许多情况下, 智能体随着时间推移变得不那么有用。例如, 如果你要求智能体准备资助提案, 智能体在截止日期后完成, 智能体就不太有帮助。

一个有趣的规划失败模式是由反思错误引起的。智能体确信它完成了任务，而实际上没有。例如，你要求智能体将50人分配到30个酒店房间。智能体可能只分配40人，并坚持认为任务已完成。

要评估智能体的规划失败，一个选择是创建规划数据集，其中每个示例是一个元组（任务，工具库）。对于每个任务，使用智能体生成K个计划。计算以下指标：

- 在所有生成的计划中，有多少是有效的？
- 对于给定任务，智能体平均需要生成多少计划才能获得有效计划？
- 在所有工具调用中，有多少是有效的？
- 无效工具被调用的频率如何？
- 有效工具被调用但参数无效的频率如何？
- 有效工具被调用但参数值不正确的频率如何？

分析智能体的输出看有无模式。智能体在哪类任务上失败更多？你有假设为什么吗？模型经常在某些工具上出错？有些工具可能对智能体来说更难使用。你可以通过更好的提示、更多示例或微调来提高智能体使用具有挑战性工具的能力。如果都失败了，你可能考虑将此工具换成更易于使用的工具。

工具失败

工具失败发生在使用了正确的工具，但工具输出错误时。一种失败模式是工具只是给出错误输出。例如，图像说明器返回错误描述，或SQL查询生成器返回错误的SQL查询。

如果智能体只生成高级计划，并且涉及翻译模块将每个计划的操作翻译成可执行命令，失败可能因翻译错误而发生。

工具失败也可能因为智能体没有访问任务所需的正确工具而发生。一个明显的例子是，当任务涉及从互联网检索当前股票价格，而智能体没有访问互联网的权限。

工具失败取决于工具。每个工具需要独立测试。始终打印出每个工具调用及其输出，以便检查和评估。如果你有翻译器，创建基准来评估它。

检测缺失工具失败需要了解应该使用哪些工具。如果你的智能体在特定领域频繁失败，这可能是因为它缺乏该领域的工具。与人类领域专家合作，观察他们会使用哪些工具。

效率

智能体可能生成有效计划，使用正确的工具完成任务，但可能效率低下。以下是一些你可能想跟踪的事项，以评估智能体的效率：

- 智能体平均需要多少步骤才能完成任务？
- 智能体平均花费多少成本才能完成任务？
- 每个操作通常需要多长时间？是否有任何操作特别耗时或昂贵？

你可以将这些指标与你的基准进行比较，基准可以是另一个智能体或人类操作员。比较AI智能体和人类智能体时，请记住人类和AI有非常不同的操作模式，所以对人类被认为有效的可能对AI来说是低效的，反之亦然。例如，访问100个网页对于一次只能访问一个页面的人类智能体可能效率低下，但对于可以同时访问所有网页的AI智能体来说是微不足道的。

在本章中，我们详细讨论了RAG和智能体系统如何运作。这两种模式通常处理超出模型上下文限制的信息。补充模型上下文处理信息的记忆系统可以显著增强其能力。让我们现在探索记忆系统是如何工作的。

记忆

记忆是指允许模型保留和利用信息的机制。记忆系统对知识丰富的应用如RAG和多步应用如智能体特别有用。RAG系统依赖记忆进行增强上下文，随着检索更多信息，这个上下文可能在多个回合中增长。智能体系统需要记忆来存储指令、示例、上下文、工具库、计划、工具输出、反思等。虽然RAG和智能体对记忆的要求更高，但记忆对任何需要保留信息的AI应用都有益。

AI模型通常有三种主要记忆机制：

内部知识

模型本身是一种记忆机制，因为它保留了从训练数据中获得的知识。这种知识是其内部知识。模型的内部知识不会改变，除非模型本身更新。模型可以在所有查询中访问这种知识。

短期记忆

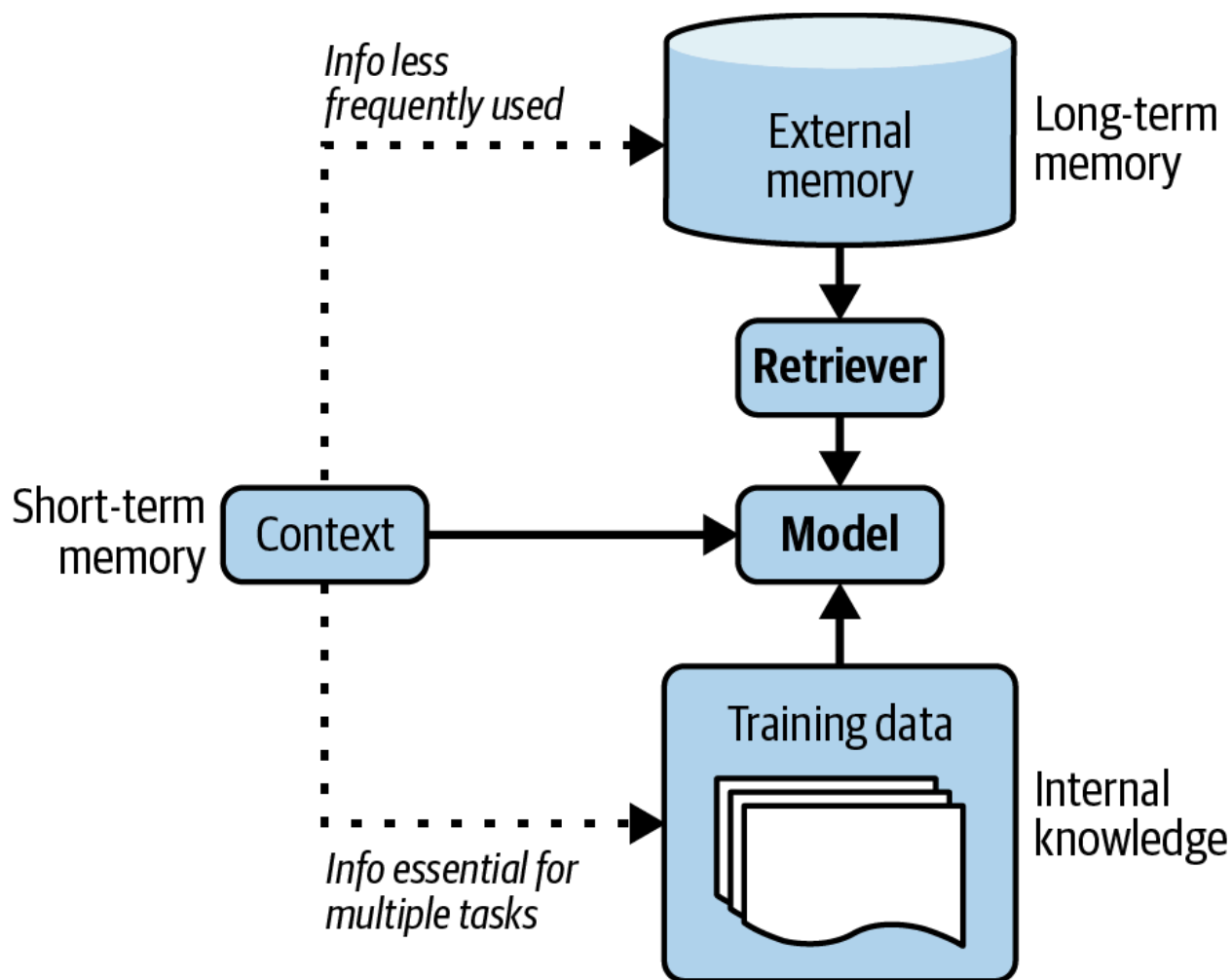
模型的上下文是一种记忆机制。对话中的先前消息可以添加到模型的上下文中，使模型能够利用它们生成未来响应。模型的上下文可以被视为其短期记忆，因为它不会在任务（查询）之间持续存在。它访问快速，但容量有限。因此，它通常用于存储对当前任务最重要的信息。

长期记忆

模型可以通过检索访问的外部数据源，如RAG系统中的数据源，是一种记忆机制。这可以被视为模型的长期记忆，因为它可以在任务之间持续存在。与模型的内部知识不同，长期记忆中的信息可以在不更新模型的情况下删除。

人类可以访问类似的记忆机制。如何呼吸是你的内部知识。除非你遇到严重麻烦，否则你通常不会忘记如何呼吸。你的短期记忆包含与你正在做的事情直接相关的信息，如你刚遇到的人的名字。你的长期记忆通过书籍、计算机、笔记等增强。

对于你的数据使用哪种记忆机制取决于其使用频率。对所有任务都必不可少的信息应通过培训或微调纳入模型的内部知识。很少需要的信息应存在于其长期记忆中。短期记忆保留用于即时的、特定上下文的信息。这三种记忆机制如图6-16所示。



记忆对人类运作必不可少。随着AI应用的发展，开发人员很快意识到记忆对AI模型也很重要。许多AI模型的记忆管理工具已经开发出来，许多模型提供商已经整合了外部记忆。用记忆系统增强AI模型有许多好处。以下仅是其中几项：

管理会话内的信息溢出

在执行任务的过程中，智能体获取大量新信息，可能超过智能体的最大上下文长度。多余信息可以存储在具有长期记忆的记忆系统中。

在会话之间持久保存信息

如果每次你想要AI教练的建议，都必须解释你的整个生活故事，这个AI教练实际上就没用了。如果AI助手不断忘记你的偏好，使用起来会很烦人。访问你的对话历史可以让智能体个性化其对你的行动。例如，当你要求书籍推荐时，如果模型记得你以前喜欢《三体》，它可以推荐类似的书籍。

提升模型的一致性

如果你问我两次主观问题，比如在1到5之间为笑话评分，如果我记得我之前的回答，我更可能给出一致的答案。同样，如果AI模型可以参考其之前的答案，它可以校准其未来答案以保持一致。

维护数据结构完整性

因为文本本质上是非结构化的，存储在基于文本的模型上下文中的数据是非结构化的。你可以将结构化数据放入上下文中。例如，你可以逐行将表格输入到上下文中，但不能保证模型会理解这应该是一个表格。能够存储结构化数据的记忆系统可以帮助维护数据的结构完整性。例如，如果你要求智能体寻找潜在销售线索，这个智能体可以利用Excel表格存储线索。智能体还可以利用队列存储要执行的操作序列。

AI模型的记忆系统通常包括两个功能：

记忆管理：管理什么信息应该存储在短期和长期记忆中。记忆检索：从长期记忆中检索与任务相关的信息。

记忆检索类似于RAG检索，因为长期记忆是外部数据源。在本节中，我将专注于记忆管理。记忆管理通常包括两个操作：添加和删除记忆。如果记忆存储有限，删除可能不是必要的。这对长期记忆可能有效，因为外部记忆存储相对便宜且易于扩展。然而，短期记忆受限于模型的最大上下文长度，因此需要策略来决定添加什么和删除什么。

长期记忆可用于存储短期记忆的溢出。这个操作取决于你想为短期记忆分配多少空间。对于给定查询，输入模型的上下文同时包括其短期记忆和从长期记忆中检索的信息。因此，模型的短期容量取决于应该为从长期记忆检索的信息保留多少上下文。例如，如果保留30%的上下文，那么模型最多可以使用70%的上下文限制作为短期记忆。当达到这个阈值时，溢出可以移至长期记忆。

像本章前面讨论的许多组件一样，记忆管理并非AI应用独有。记忆管理一直是所有数据系统的基石，已经开发了许多策略来有效使用记忆。

最简单的策略是FIFO，先进先出。首先添加到短期记忆的将是第一个被移动到外部存储的。随着对话变长，像OpenAI这样的API提供商可能会开始删除对话的开头。像LangChain这样的框架可能允许保留最后N条消息或最后N个令牌。在长对话中，这种策略假设早期消息对当前讨论的相关性较低。然而，这一假设可能严重错误。在某些对话中，最早的消息可能包含最多信息，特别是当早期消息陈述对话目的时。虽然FIFO实现简单，但它可能导致模型丢失重要信息。

更复杂的策略涉及删除冗余。人类语言包含冗余以增强清晰度并补偿潜在的误解。如果有一种方法可以自动检测冗余，记忆占用空间将显著减少。

删除冗余的一种方法是使用对话摘要。这个摘要可以使用相同或另一个模型生成。摘要，连同跟踪命名实体，可以带你走很远。Bae等(2022)更进一步。获得摘要后，作者希望通过将记忆与摘要遗漏的关键信息连接起来构建新记忆。作者开发了一个分类器，对于记忆中的每个句子和摘要中的每个句子，确定是只添加一个，两个都添加，还是都不添加到新记忆中。

另一方面，Liu等(2023)使用了反思方法。每个操作后，要求智能体做两件事：

1. 反思刚刚生成的信息。
2. 确定这个新信息是否应该插入到记忆中，应该与现有记忆合并，还是应该替换其他信息，特别是如果其他信息已过时并与新信息矛盾。

当遇到矛盾信息时，有些人选择保留较新的信息。有些人要求AI模型判断保留哪一个。如何处理矛盾取决于用例。有矛盾可能导致智能体感到困惑，但也可以帮助它从不同角度获取信息。

总结

鉴于RAG和智能体的受欢迎程度，早期读者提到这是他们最期待的章节。

本章从两者之间首先出现的模式RAG开始。许多任务需要广泛的背景知识，通常超过模型的上下文窗口。例如，代码副驾可能需要访问整个代码库，研究助手可能需要分析多本书。RAG最初是为克服模型上下文限制而开发的，还能更有效地使用信息，在降低成本的同时提高响应质量。从基础模型的早期，就明显看出RAG模式对广泛应用非常有价值，此后它已经在消费者和企业用例中迅速采用。

RAG采用两步过程。它首先从外部记忆中检索相关信息，然后使用这些信息生成更准确的响应。RAG系统的成功取决于其检索器的质量。基于术语的检索器，如Elasticsearch和BM25，实现起来轻量得多，可以提供强大的基线。基于嵌入的检索器计算强度更大，但有可能超越基于术语的算法。

基于嵌入的检索由向量搜索提供支持，这也是许多核心互联网应用如搜索和推荐系统的支柱。为这些应用开发的许多向量搜索算法可用于RAG。

RAG模式可视为智能体的特殊情况，其中检索器是模型可以使用的工具。这两种模式都允许模型绕过其上下文限制并保持更新，但智能体模式的功能远不止于此。智能体由其环境和可以访问的工具定义。在AI驱动的智能体中，AI是规划器，分析给定任务，考虑不同解决方案，并选择最有希望的一个。复杂任务可能需要多个步骤解决，这需要强大的模型来规划。可以用反思和记忆系统增强模型的规划能力，帮助它跟踪进度。

你给模型的工具越多，模型的能力就越强，使其能够解决更具挑战性的任务。然而，智能体越自动化，其失败可能越灾难性。工具使用使智能体面临第5章讨论的许多安全风险。为使智能体在现实世界中工作，需要建立严格的防御机制。

RAG和智能体都处理大量信息，通常超过底层模型的最大上下文长度。这需要引入记忆系统来管理和使用模型拥有的所有信息。本章以简短讨论这个组件的样子结束。

RAG和智能体都是基于提示的方法，它们仅通过输入影响模型的质量，而不修改模型本身。虽然它们能够实现许多令人难以置信的应用，但修改底层模型可以开拓更多可能性。如何做到这一点将是下一章的主题。